

IC TRAINING CENTER VIET NAM
FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE



FINAL PROJECT
TIMER IP DESIGN SPECIFICATION

Student: NGUYEN DUC TRI

Class: IC37

Instructor: VU NGOC TUYEN

Contents

LIST OF TABLES	5
LIST OF FIGURES	6
1. OVERVIEW	7
1.1. Introduction.....	7
1.2. Main Features.....	7
1.2.1. Timer function.....	7
1.2.2. APB interface.....	8
1.2.3. Counter control	8
1.2.4. Interrupt function	8
1.2.5. Debug support.....	9
1.2.6. Advanced APB features	9
1.3. Block Diagram	10
1.4. Interface Signals.....	12
1.4.1. Interface overview.....	12
1.4.2. Signal description.....	13
1.4.3. Interface conventions	14
2. REGISTER SPECIFICATION	15
2.1. Register Summary.....	15
2.2. TCR – Timer Control Register.....	15
2.3. TDR0 & TDR1 – Timer Data Register	17
2.4. TCMP0 & TCMP1 – Timer Compare Register	19
2.5. TIER – Timer Interrupt Enable Register.....	20
2.6. TISR – Timer Interrupt Status Register	20
2.7. THCSR – Timer Halt Control Status Register	21
3. FUNCTIONAL DESCRIPTION	23
3.1. Timer Top Architecture	23
3.2. APB Slave.....	24
3.2.1. SETUP & ACCESS phase	25
3.2.2. Request capture	25

3.2.3.	One-cycle wait-state.....	25
3.2.4.	Read / write completion	25
3.2.5.	Write commit	25
3.2.6.	PSLVERR response	26
3.2.7.	Back-to-back transfers	26
3.3.	Register Block.....	26
3.3.1.	Address decode	26
3.3.2.	Register read mux	27
3.3.3.	PSTRB byte merge.....	27
3.3.4.	RAZ/WI handling	27
3.3.5.	TCR candidate value & error detection	27
3.3.6.	Register command pulses.....	28
3.3.7.	Timer disable pulse	28
3.3.8.	Access error logic.....	29
3.4.	Counter Control Block.....	29
3.4.1.	Default counting mode.....	30
3.4.2.	Divider bypass mode.....	30
3.4.3.	Programmable divider mode	30
3.4.4.	Internal divider counter	31
3.4.5.	Count_en generation	31
3.4.5.1.	First count_en generation.....	31
3.4.5.2.	Second count_en generation	32
3.4.6.	Debug halt and divider phase preservation	33
3.5.	Counter.....	34
3.5.1.	Counter increment.....	35
3.5.2.	Low-to_high carry.....	35
3.5.3.	64-bit overflow and wrap.....	36
3.5.4.	TDR software write.....	36
3.5.5.	Timer disable clear.....	36
3.5.6.	Counter update priority	37
3.6.	Interrupt.....	37
3.6.1.	64-bit compare	38

3.6.2.	Interrupt pending status.....	39
3.6.3.	Interrupt masking	39
3.6.4.	RW1C clear.....	39
3.6.5.	Clear / Match priority.....	40
3.6.6.	Interrupt reassertion	40
3.6.7.	Counter operation after interrupt.....	40
3.7.	Debug Halt Operation	40
3.7.1.	Counter operation after interrupt.....	41
3.7.2.	Counter operation after interrupt.....	41
3.7.3.	Counter operation after interrupt.....	41
3.8.	Reset Behavior	42
4.	VERIFICATION SUMMARY	44
4.1.	Verification Environment	44
4.2.	Verification Strategy	45
4.3.	Testcases Summary.....	45
4.4.	Key Verification Scenarios	46
4.5.	Regression Result.....	52
4.6.	Code Coverage Result.....	53
5.	DESIGN CONCLUSION	55

LIST OF TABLES

Table 1-1 Timer IP Functional Block Description.....	11
Table 1-2 Main Inter-Block Signals.....	11
Table 1-3 Timer IP Interface Signal Description.....	13
Table 2-1 Timer Register Specification	15
Table 2-2 TCR description.....	16
Table 2-3 THCSR register operation description.....	22
Table 3-1 Counter_control mode setting	30
Table 3-2 Counter-update events priority of timer_counter	37
Table 3-3 Reset value of timer IP	42
Table 4-1 Total testcases.....	46

LIST OF FIGURES

Figure 1-1 Timer IP Top-Level Block Diagram	10
Figure 2-1 The simulation of the TCR signal field.....	16
Figure 2-2 Logic diagram TCR register.....	17
Figure 2-3 Logic diagram TDR register	18
Figure 2-4 Logic diagram TCMP register.....	19
Figure 2-5 Logic diagram TIER register.....	20
Figure 2-6 Logic diagram TISR register.....	21
Figure 2-7 Logic diagram THCSR register.....	22
Figure 3-1 Advanced Level block diagram TIMER_TOP.....	23
Figure 3-2 Logic diagram for Timer_apb_slave	24
Figure 3-3 Logic diagram timer_disable_pulse	28
Figure 3-4 Logic diagram access_error	29
Figure 3-5 Logic diagram timer_counter_control.....	32
Figure 3-6 Logic diagram count_en (second version)	32
Figure 3-7 Logic diagram timer_counter	34
Figure 3-8 Logic diagram timer_interrupt	38
Figure 4-1Timer_top structure	44
Figure 4-2Timer_top - Asynchronous active-low reset waveform.....	47
Figure 4-3Timer_top - Configure, count, interrupt and clear waveform	47
Figure 4-4 APB Slave - Back-to-back write transfers waveform.....	47
Figure 4-5 APB Slave - Error response & no register update waveform.....	48
Figure 4-6 APB Slave - Write/Read with one wait state waveform	48
Figure 4-7 Register - address decode and read mux waveform.....	48
Figure 4-8 Register - TCR byte0 write waveform	49
Figure 4-9 Register - TDR0 byte write waveform.....	49
Figure 4-10 Register - Timer_disable_pulse waveform.....	49
Figure 4-11 Counter - update priority waveform.....	49
Figure 4-12 Counter - Low-to-high waveform	50
Figure 4-13 Counter - Overflow & wrap around waveform	50
Figure 4-14 Counter control - Default mode waveform	50
Figure 4-15 Counter control - case div_val = 8 waveform	50
Figure 4-16 Counter control - case div_val = 1,2,3 waveform	51
Figure 4-17 Halt_req & halt_ack waveform	51
Figure 4-18 Counter control - halt freeze waveform	51
Figure 4-19 Interrupt - No stop counter waveform.....	52
Figure 4-20 Interrupt - Match pending, masking and RW1C clear waveform	52
Figure 4-21 Interrupt - RW1C clear priority waveform.....	52
Figure 4-22Coverage check 1	54
Figure 4-23Coverage check 2	54
Figure 4-24 Coverage check 3	54

1. OVERVIEW

1.1. Introduction

The Timer IP is a programmable timer peripheral designed to provide accurate timing and interrupt generation functions in a digital system. It operates as an Advanced Peripheral Bus (APB) slave device, allowing software or a processor to configure and monitor the timer through memory-mapped registers. By integrating with the APB bus architecture, the Timer IP can be easily incorporated into a System-on-Chip (SoC) design while maintaining a simple and efficient programming interface.

The primary function of the Timer IP is to implement a 64-bit count-up timer that can operate at the system clock frequency or at a programmable counting rate through a configurable divider. The timer continuously compares the current counter value with a user-defined compare value. When both values match, an interrupt request can be generated to notify the processor or other hardware modules that a predefined timing event has occurred. This mechanism allows the timer to be used for periodic interrupts, timeout detection, event scheduling, software delay generation, and many other embedded system applications.

To improve flexibility and software controllability, the Timer IP provides a set of memory-mapped control and status registers for configuring timer operation, interrupt management, compare values, and debug behavior. In the Advanced Level implementation, the design additionally supports one-cycle APB wait-state insertion, APB error response, byte write access using write strobes, and timer halt operation during debug mode, making the IP more representative of industrial timer peripherals.

This document describes the Timer IP architecture, interface signals, register specifications, functional behavior, and design implementation details. It serves as the design specification for RTL development and provides the functional reference for verification and validation throughout the project.

1.2. Main Features

The Timer IP provides a comprehensive set of hardware features to support timer operation, APB communication, interrupt generation, and debug control. The design is organized into several functional groups to improve readability and to clearly separate the responsibilities of each subsystem. This organization also simplifies RTL implementation, functional verification, and future maintenance by allowing each function to be independently designed, verified, and documented. The major features supported by the Timer IP are described as follows.

1.2.1. Timer function

The Timer Function is the fundamental component of the Timer IP and is responsible for providing timing services for the system. It implements a 64-bit free-running count-up timer that continuously increments while the timer is enabled. The counter can be initialized by software through dedicated timer data registers and compared with a programmable compare value to determine when a timing event occurs. The timer continues operating after a compare match or counter overflow unless it is explicitly disabled or

halted, ensuring continuous timing capability for long-running applications. The Timer Function supports the following features:

- 64-bit free-running count-up counter.
- Software programmable timer initialization.
- 64-bit compare operation.
- Continuous counting after compare match.
- Continuous counting after counter overflow.
- Active-low asynchronous reset.

1.2.2. APB interface

The APB Interface provides the communication channel between the Timer IP and the APB master. All timer configurations and status information are accessed through memory-mapped registers using the AMBA APB protocol. This interface allows software to configure timer operation, read timer status, initialize counter values, and control interrupt behavior without directly accessing internal hardware logic. The APB Interface supports the following features:

- AMBA APB slave interface.
- 12-bit address bus.
- 32-bit read/write data bus.
- Memory-mapped register access.
- Read and write transactions.
- Reserved address and reserved field handling following the Read-As-Zero / Write-Ignored (RAZ/WI) policy.

1.2.3. Counter control

The Counter Control function determines how the timer counter progresses during operation. Instead of modifying the clock source, this block controls the counting interval by generating internal counting events according to user configuration. Two operating modes are provided to satisfy different timing requirements while maintaining a synchronous design methodology. The Counter Control supports the following features:

- Timer enable and disable control.
- Default counting mode using the system clock.
- Control mode with programmable counting speed.
- Programmable counting interval from divide-by-1 to divide-by-256.
- Divider configuration through DIV_EN and DIV_VAL.
- Internal counting control without generating a derived clock.

1.2.4. Interrupt function

The Interrupt Function allows the Timer IP to notify the processor whenever a predefined timer event occurs. A timer interrupt is generated by comparing the current counter value with the programmed compare value. To provide flexible software control, the interrupt mechanism includes dedicated enable,

status, and clear functions. The interrupt status remains asserted until software explicitly clears it, allowing reliable interrupt servicing. The Interrupt Function supports the following features:

- Programmable timer interrupt generation.
- 64-bit compare operation.
- Interrupt enable control.
- Interrupt status indication.
- Interrupt status supports Read/Write-1-to-Clear (RW1C) operation.
- Interrupt pending until software clears the interrupt status.

1.2.5. Debug support

The Debug Support function provides hardware assistance during software debugging and system validation. When the processor enters debug mode, the timer can be temporarily suspended to prevent timer progression while debugging software execution. Once the halt request is released, the timer resumes normal operation without affecting the remaining timer configuration. The Debug Support supports the following features:

- Timer halt request.
- Timer halt acknowledge.
- Timer suspend operation during debug mode.
- Timer resume after halt request is released.

1.2.6. Advanced APB features

To satisfy the requirements of the Advanced Level Final Project, the Timer IP implements several additional APB features beyond a conventional timer peripheral. These features improve protocol compliance, increase software flexibility, and provide more realistic hardware behavior similar to industrial peripheral IPs. They also introduce additional verification scenarios for RTL validation and functional coverage. The Advanced APB Features include:

- One-cycle APB wait-state insertion.
- APB slave error response.
- Byte write access using `tim_pstrb[3:0]`.
- Illegal divider configuration detection.
- Protection against divider reconfiguration while the timer is operating.

1.3. Block Diagram

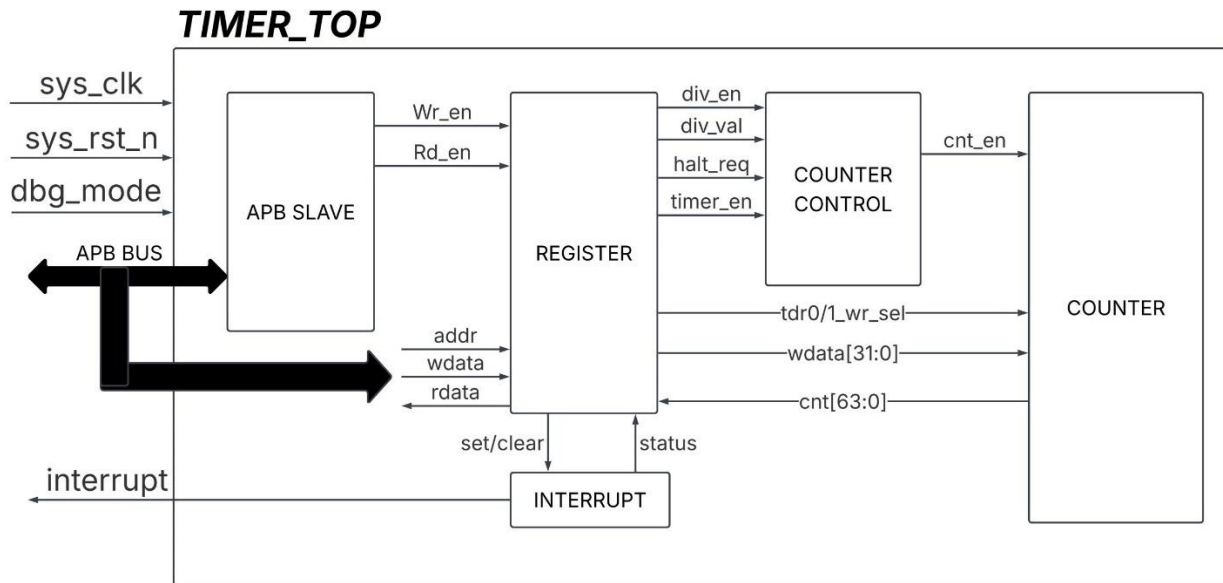


Figure 1-1 Timer IP Top-Level Block Diagram

The Timer IP is structured into five principal functional modules: the APB Slave, Register, Counter Control, Counter, and Interrupt units. These modules operate in coordination to deliver a register-configurable timer mechanism, supporting divided counting modes, interrupt generation, APB error management, byte-level write access, and debug halt functionality. Figure 1-1 depicts the top-level architecture, highlighting the primary data flows and control paths interconnecting the functional modules.

The APB Slave block constitutes the bus-facing interface of the Timer IP. It is responsible for detecting the APB setup and access phases, capturing the transfer address, write data, and byte strobes, inserting a single wait-state cycle, and generating the APB completion response. In addition, it produces the read data, ready, and slave-error outputs. A write transaction is committed to the register block only after the APB transfer has been successfully completed and no access error has been identified.

The Register block implements the memory-mapped register set, including TCR, TDR0, TDR1, TCMP0, TCMP1, TIER, TISR, and THCSR. It performs address decoding, byte-mask generation, write-data merging, reserved-field handling, and read-data multiplexing. The block also validates TCR programming requests and generates an access error when an invalid divisor value is written or when the divisor configuration is modified while the timer is operating.

The Counter Control block determines when the 64-bit timer counter is permitted to increment. It supports the default counting mode, programmable divided counting mode, timer disable handling, and debug halt operation. An internal 8-bit divider counter generates the required counting enable pulse according to the programmed divisor value. When both `dbg_mode` and `halt_req` are asserted, the counter and internal divider state are held until the halt request is removed.

The Counter block incorporates the 64-bit count-up register. Its value can be incremented by the Counter Control block, updated through software writes to TDR0 or TDR1, or cleared when the timer is disabled. Both the current and next counter values are supplied to the Register and Interrupt blocks to enable software readback and compare-event detection.

The Interrupt block compares the effective counter value against the programmed 64-bit compare value. Upon detection of a valid compare event, the sticky interrupt status bit is asserted and remains active until explicitly cleared through a write-one-to-clear operation in TISR. The external timer interrupt is generated by combining the interrupt status with the interrupt-enable bit through masking logic.

Table 1-1 Timer IP Functional Block Description

Block	Main responsibility
APB Slave	Detect APB phases, latch requests, generate one wait state, complete read/write transfers, and generate tim_pready and tim_pslverr.
Register	Decode addresses, store configuration registers, implement byte access, generate read data, and detect invalid TCR accesses.
Counter Control	Generate counter enable according to timer_en, div_en, div_val, and debug halt conditions.
Counter	Store and update the 64-bit timer count value and support software writes through TDR0/TDR1.
Interrupt	Compare timer count with compare value, maintain sticky interrupt status, implement RW1C clear, and generate tim_int.

Table 1-2 Main Inter-Block Signals

Source	Destination	Signals	Purpose
APB Slave	Register	reg_addr, reg_wdata, reg_pstrb, wr_commit, rd_complete	Transfer APB request information
Register	APB Slave	reg_rdata, access_error	Return read data and transaction error
Register	Counter Control	timer_en, div_en, div_val, halt_req	Configure timer operating mode

Source	Destination	Signals	Purpose
Counter Control	Register	halt_ack	Indicate accepted halt condition
Counter Control	Counter	count_en, timer_disable_pulse	Control counter update and clear
Register	Counter	TDR write enables and merged data	Software-load the 64-bit count value
Counter	Register	counter_value[63:0]	Provide TDR readback
Register	Interrupt	compare_value, int_en, int_clear	Configure and clear interrupt
Counter	Interrupt	counter_value or counter_next_value	Detect compare event
Interrupt	Register	int_st	Provide TISR status
Interrupt	External	tim_int	Timer interrupt output

1.4. Interface Signals

1.4.1. Interface overview

The Timer IP is implemented as an APB slave peripheral, interfacing with an APB master via a 12-bit address bus and a 32-bit data bus. All sequential operations within the Timer IP are synchronized to the rising edge of sys_clk, while sys_rst_n provides an active-low asynchronous reset to initialize and maintain the integrity of the internal state elements.

The APB interface consists of address, control, write-data, read-data, byte-strobe, ready, and error-response signals. In the Advanced Level implementation, the Timer IP supports one wait-state cycle, byte-write access through tim_pstrb[3:0], and APB error reporting through tim_pslverr. The tim_int output indicates a pending and enabled timer interrupt, while the dbg_mode input allows the timer counter to be halted when a valid halt request is programmed through the Timer Halt Control and Status Register.

The top-level module shall be named timer_top, and all interface signals listed in Table 1-3 shall be implemented with the specified direction and width.

1.4.2. Signal description

Table 1-3 Timer IP Interface Signal Description

Signal	Direction	Width	Description
sys_clk	Input	1	System clock input. All synchronous operations of the Timer IP are performed on the rising edge of this clock.
sys_rst_n	Input	1	Active-low asynchronous reset. When asserted Low, internal registers, the counter, divider state, interrupt status, and APB control state are reset to their specified initial values.
tim_psel	Input	1	APB peripheral select signal. High indicates that the Timer IP is selected for an APB transfer.
tim_penable	Input	1	APB enable signal. Low during the setup phase and High during the access phase of an APB transfer.
tim_pwrite	Input	1	APB transfer direction. High indicates a write transfer; Low indicates a read transfer.
tim_paddr	Input	12	APB address bus used to select a Timer IP register or reserved address location.
tim_pwdata	Input	32	APB write-data bus. Contains the data to be written during an APB write transfer.
tim_pstrb	Input	4	APB byte-write strobes. Each bit enables one byte lane of tim_pwdata. This signal is used only for write transfers.
tim_prdata	Output	32	APB read-data bus. Returns the selected register value during a valid APB read transfer. Reserved addresses and reserved fields return zero.

Signal	Direction	Width	Description
tim_pready	Output	1	APB ready signal. Indicates completion of the current APB transfer. In the Advanced Level implementation, the slave inserts exactly one wait-state cycle before asserting this signal.
tim_pslverr	Output	1	APB slave-error response. Asserted during the completion cycle of an invalid or prohibited APB access.
tim_int	Output	1	Timer interrupt output. Asserted when the interrupt status is pending and the interrupt-enable bit is set.
dbg_mode	Input	1	Debug-mode indication. When asserted together with THCSR.halt_req, the timer counter and divider operation are halted.

1.4.3. Interface conventions

All input signals shall be sampled with respect to the rising edge of sys_clk, except for sys_rst_n, which is an active-low asynchronous reset. APB address, control, write-data, and byte-strobe signals shall remain stable throughout the access phase until tim_pready is asserted.

An APB transfer is considered complete only when *tim_psel*, *tim_penable*, and *tim_pready* are asserted simultaneously. Write operations update the target register exclusively upon successful transfer completion and are not committed if *tim_pslverr* is asserted.

For write transfers, each bit of tim_pstrb[3:0] controls the corresponding byte lane of tim_pwdata[31:0]. Bytes associated with deasserted strobe bits shall retain their previous values. Reserved register fields and reserved addresses shall be read as zero and shall ignore write operations.

The tim_int output is a level-sensitive interrupt signal. It remains asserted while the interrupt status is pending and interrupt generation is enabled. The dbg_mode input enables the debug-halt mechanism but does not halt the timer unless the halt_req field of THCSR is also asserted.

2. REGISTER SPECIFICATION

2.1. Register Summary

The Timer IP provides a set of memory-mapped registers that configure timer operation, define counter and compare values, control interrupt behavior, and support debug-halt functionality. These registers are accessed via the APB interface and are assigned fixed addresses within the Timer IP register space. Each register or field is associated with a specific access attribute, enabling software configuration while preserving hardware-controlled status and reserved fields. The complete register map is summarized in the following table.

Access attribute conventions:

- RW (Read/Write): The field can be read and modified by software.
- RO (Read-Only): The field is readable by software and updated exclusively by hardware.
- RW1C (Read/Write-1-to-Clear): Writing a logic ‘1’ clears the corresponding status bit, while writing ‘0’ leaves it unchanged.
- RAZ/WI (Read-As-Zero/Write-Ignored): Applied to reserved fields or addresses, which always return zero when read and ignore software writes.
- RW/HW (Read/Write with Hardware Update): The field can be written by software and may also be updated by hardware during normal Timer IP operation.

Table 2-1 Timer Register Specification

Address	Register	Access	Reset	Meaning
0x00	TCR	RW	0x0000_0100	Timer/divider control
0x04	TDR0	RW/HW	0x0000_0000	Counter [31:0]
0x08	TDR1	RW/HW	0x0000_0000	Counter [63:32]
0x0C	TCMP0	RW	0xFFFF_FFFF	Compare [31:0]
0x10	TCMP1	RW	0xFFFF_FFFF	Compare [63:32]
0x14	TIER	RW	0x0000_0000	Interrupt enable
0x18	TISR	RW1C/HW	0x0000_0000	Interrupt pending
0x1C	THCSR	RW/RO	0x0000_0000	Halt request/ack
Others	Reserved	RAZ/WI	0	Read zero/write ignored

2.2. TCR – Timer Control Register

The Timer Control Register (*TCR*) configures the primary operating mode of the Timer IP, including timer enable control, divider enable control, and programmable divider selection. The *DIV_VAL* field accepts legal values from 0 to 8, corresponding to supported counting intervals, while values from 9

to 15 are prohibited. When the timer is enabled, any transaction attempting to modify *DIV_EN* or *DIV_VAL* is rejected to prevent alteration of the counting period during active operation. Rewriting the same divider value is not considered a configuration change and therefore does not trigger an error. For an illegal *TCR* write, the APB transfer completes normally with *PREADY*=1, but *PSLVERR* is asserted in the completion cycle and no *TCR* field is updated. Furthermore, if a single transaction attempts to disable the timer while simultaneously modifying the divider configuration, the entire transaction is atomically rejected, ensuring that no partial update occurs.

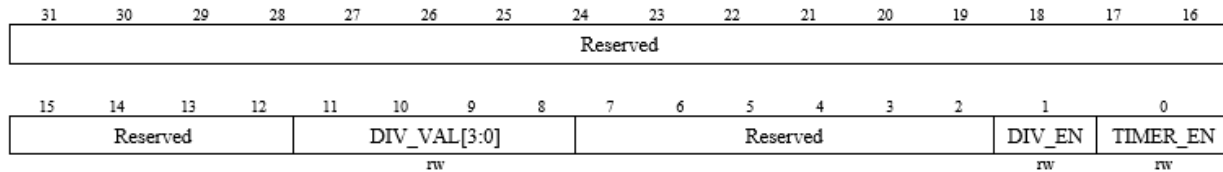


Figure 2-1 The simulation of the TCR signal field

Table 2-2 TCR description

Bits	Name	Access	Reset	Description
[31:12]	Reserved	RAZ/WI	0	Read zero, writes ignored
[11:8]	DIV_VAL	RW	4'h1	Divider selection 0–8
[7:2]	Reserved	RAZ/WI	0	Read zero, writes ignored
[1]	DIV_EN	RW	0	Enable programmable divider
[0]	TIM_EN	RW	0	Enable timer counting

The TCR access-error logic evaluates the candidate register value after applying the selected byte lanes from *PSTRB*, ensuring that only fields explicitly targeted by the write strobe are considered in legality checks.

The logic identifies three primary error conditions: an invalid *DIV_VAL* greater than 8, a modification of *DIV_EN* while the timer is active, and a modification of *DIV_VAL* during active operation. These conditions are combined to generate the TCR illegal-access indication. When a completed write targets TCR and the candidate configuration violates any of these rules, the Register block asserts *access_error*. The APB Slave then translates this condition into *PSLVERR* during the final *ACCESS* cycle and blocks *wr_commit*, thereby preserving the previous register contents. An accompanying Access Error logic diagram should be placed immediately after this description to illustrate candidate-value evaluation, illegal-condition detection, and transaction-rejection flow.

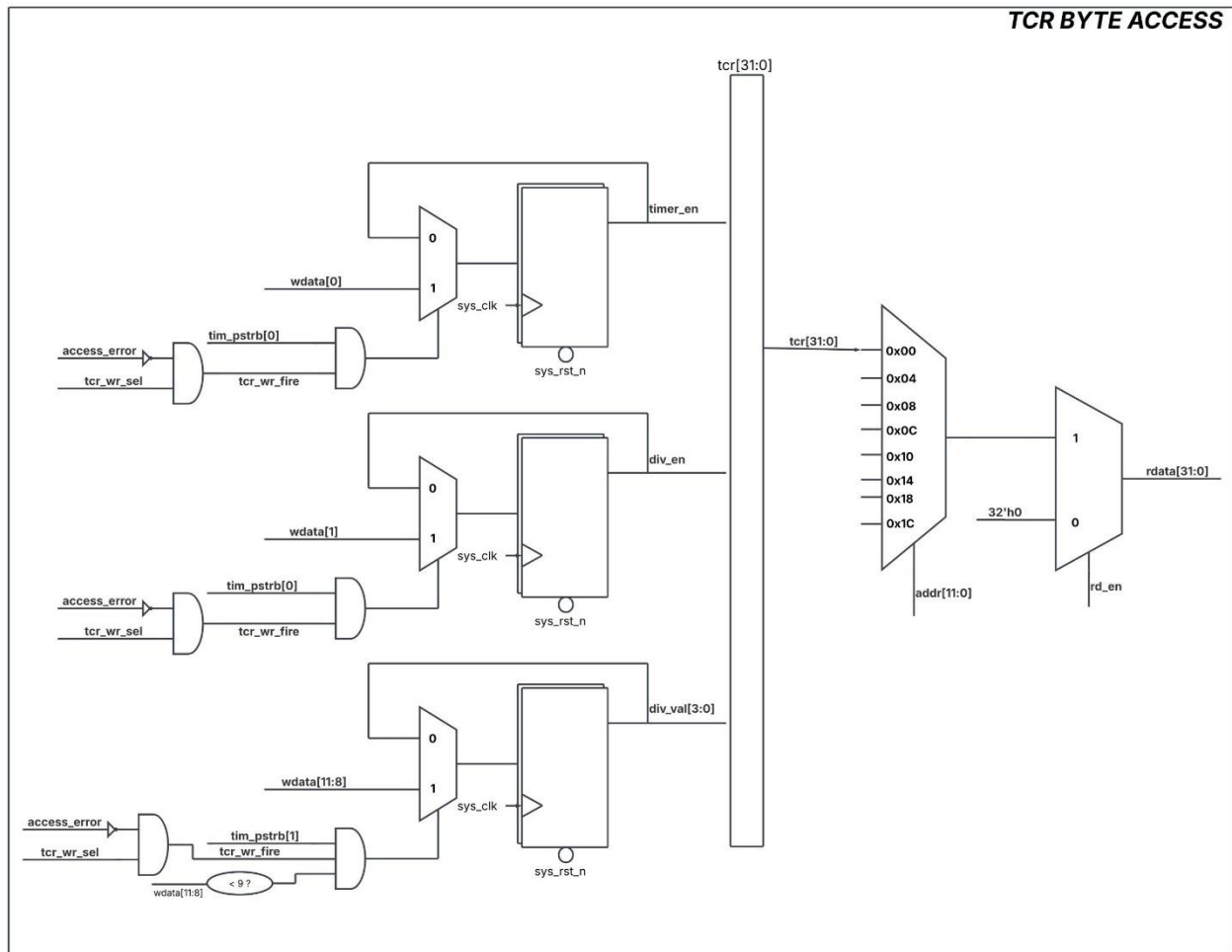


Figure 2-2 Logic diagram TCR register

2.3. TDR0 & TDR1 – Timer Data Register

The Timer Data Registers (TDR0 and TDR1) together form a single 64-bit timer counter rather than two independent counters. The complete counter value is defined as:

$$\text{count_value}[63:0] = \{\text{TDR1}, \text{TDR0}\}$$

where TDR0 holds the lower 32 bits and TDR1 holds the upper 32 bits of the counter. During normal operation, the hardware updates the entire 64-bit value as a unified counter, ensuring that any carry generated by an overflow in TDR0 is correctly propagated into TDR1. Consequently, when the counter reaches 64'hFFFF_FFFF_FFFF_FFFF, the next increment naturally wraps to 64'h0000_0000_0000_0000.

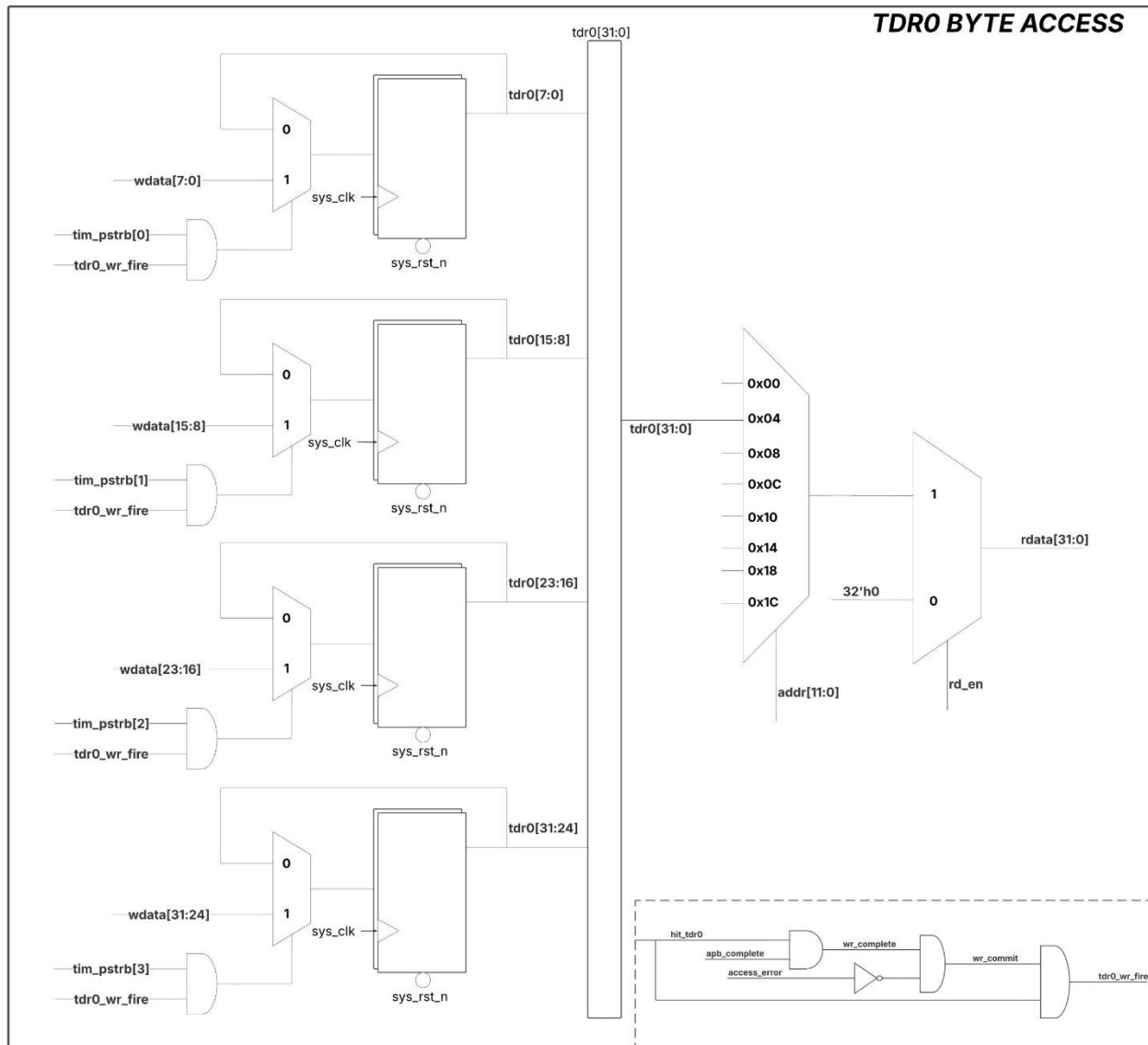


Figure 2-3 Logic diagram TDR register

Software may program either the lower or upper 32-bit portion independently through APB writes. If a software write to TDR coincides with a hardware count event in the same cycle, the software write takes precedence to prevent the programmed value from being overwritten. The counter is cleared only when TIM_EN transitions from 1 to 0; once the timer is disabled, software may still update TDR0 or TDR1, and the written value is retained until a subsequent operation modifies the counter. This mechanism allows the counter to be initialized by software while disabled, ensuring proper configuration before normal counting begins.

2.4. TCMP0 & TCMP1 – Timer Compare Register

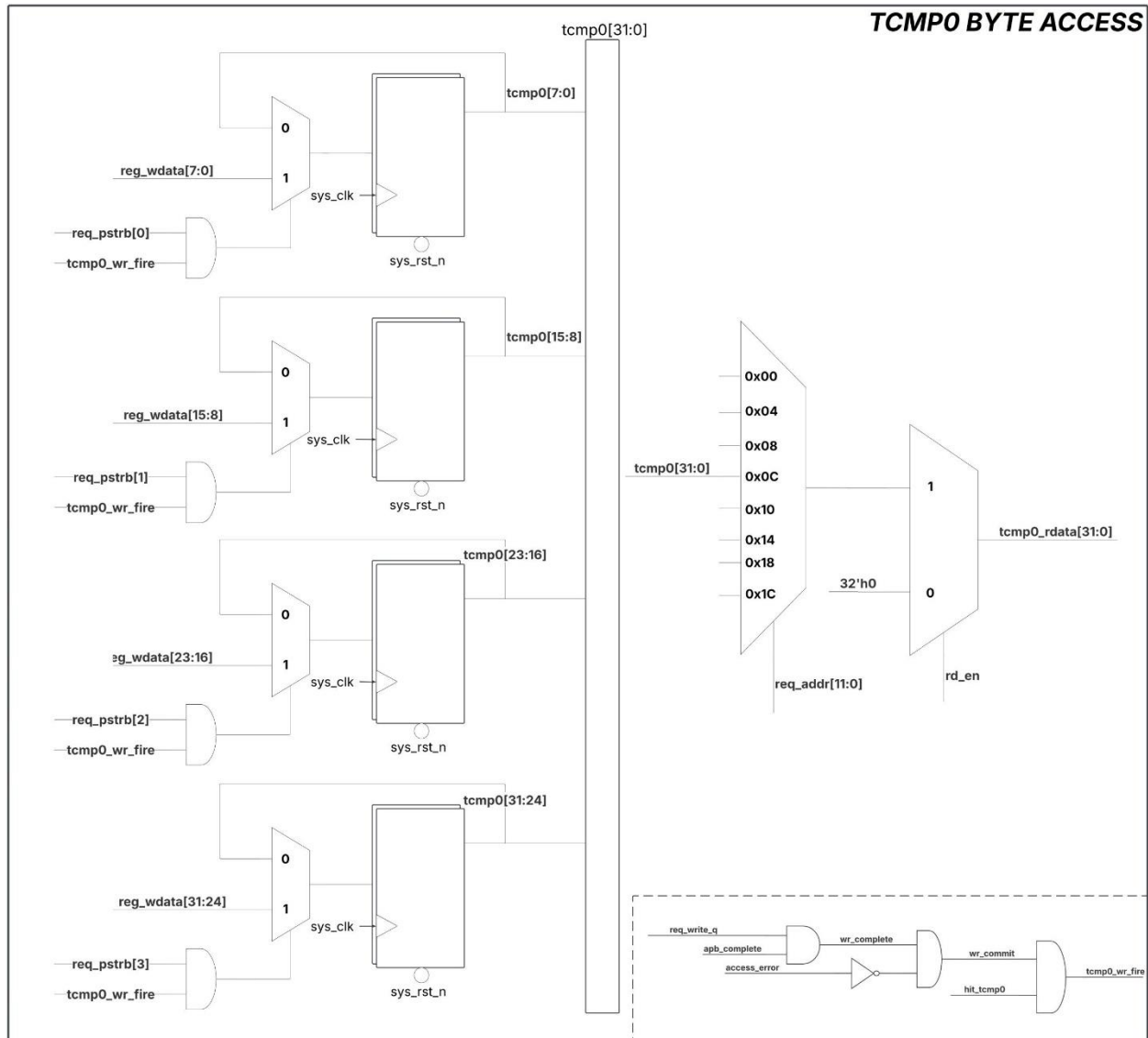


Figure 2-4 Logic diagram TCMP register

The Timer Compare Registers (TCMP0 and TCMP1) jointly define a single 64-bit compare value utilized by the interrupt generation logic. The complete compare value is expressed as:

$$compare_value[63:0] = \{TCMP1, TCMP0\}$$

where TCMP0 stores the lower 32 bits and TCMP1 stores the upper 32 bits of the programmed threshold. Following reset, the combined compare value is initialized to 64'hFFFF_FFFF_FFFF_FFFF. During timer operation, the interrupt logic evaluates the full 64-bit counter against the full 64-bit compare value, rather

than checking only the lower word. A compare match is therefore valid only when both the upper and lower halves are equal, ensuring correct interrupt behavior across the entire 64-bit counter range.

2.5. TIER – Timer Interrupt Enable Register

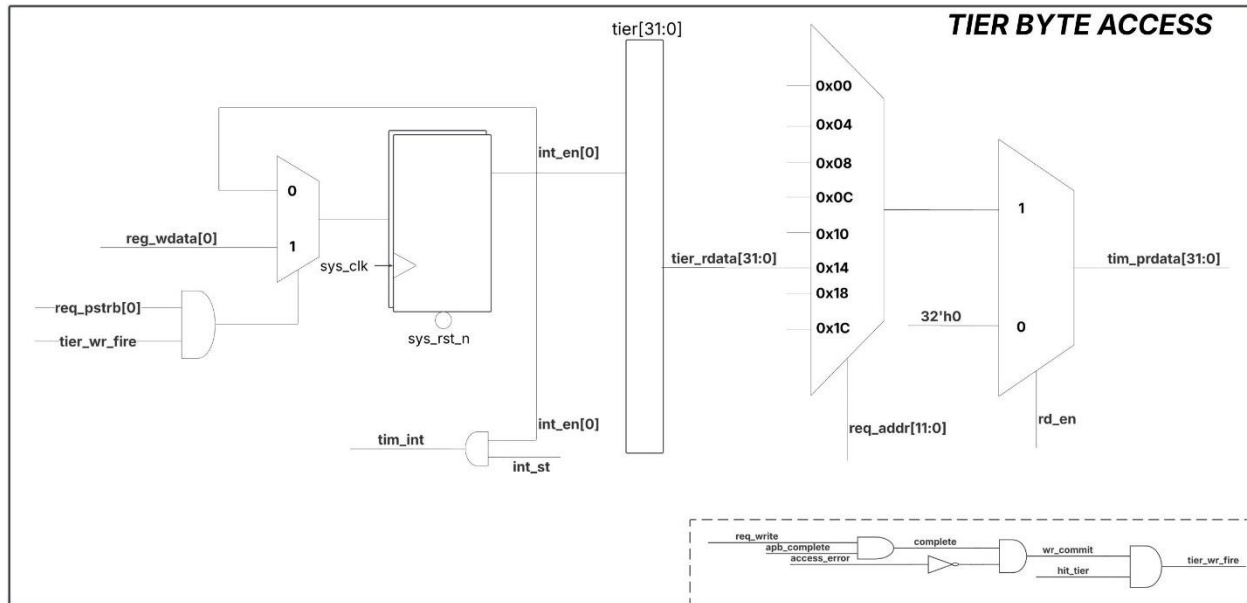


Figure 2-5 Logic diagram TIER register

The Timer Interrupt Enable Register (TIER) determines whether a pending timer interrupt is propagated to the external interrupt output. Bit 0 (INT_EN) serves as the interrupt-enable field, while bits [31:1] are reserved and follow the defined reserved-field conventions. The INT_EN field functions solely as a mask for the external tim_int signal and does not inhibit the interrupt-pending status in TISR from being set when a valid compare condition occurs. As a result, the interrupt status may become pending even when INT_EN=0; in this case, tim_int remains deasserted until the interrupt is enabled. If INT_EN is subsequently asserted while the pending status is still active, the interrupt output is driven accordingly. This behavior aligns with the final verification plan, which specifies that INT_EN only masks tim_int, and is explicitly validated by the tc_interrupt_check testcase.

2.6. TISR – Timer Interrupt Status Register

The Timer Interrupt Status Register (TISR) records the pending state of the timer interrupt. Bit 0 (INT_ST) is implemented as an RW1C/HW field, meaning it can be set by hardware and cleared by software. Hardware asserts INT_ST whenever the full 64-bit timer count matches the programmed 64-bit compare value.

$$\text{count_value}[63:0] = \text{compare_value}[63:0]$$

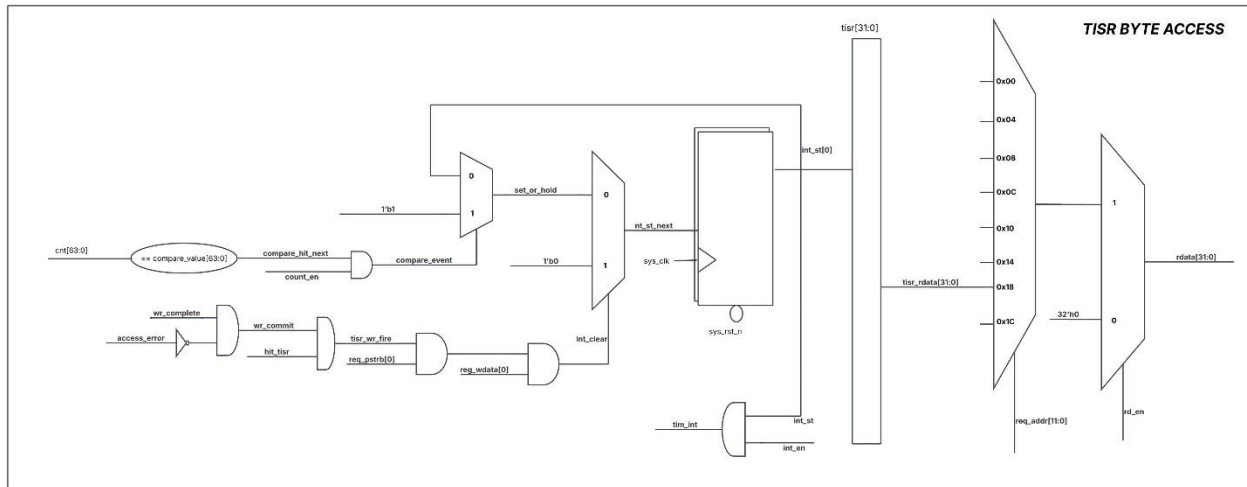


Figure 2-6 Logic diagram TISR register

This behavior is independent of the interrupt-enable state, so INT_ST may still be asserted when INT_EN=0; in such cases, only the external tim_int output remains masked. A software write to the Timer Data Registers may also cause INT_ST to be set if the resulting 64-bit counter value equals the compare value, even while timer_en=0. Software clears the pending status by performing a valid write to TISR with byte lane 0 selected and PWDATA[0]=1, consistent with the RW1C convention.

If a clear request and a compare match occur in the same clock cycle, the clear operation takes precedence. However, if the counter and compare values remain equal in the subsequent cycle, hardware may reassert INT_ST according to the defined match behavior. This interaction between hardware set, software clear, interrupt masking, and compare persistence is a fundamental aspect of the Timer IP interrupt mechanism and is explicitly covered in the final verification baseline.

2.7. THCSR – Timer Halt Control Status Register

The Timer Halt Control and Status Register (THCSR) provides software control and status reporting for the timer debug-halt function. Bit 0 (HALT_REQ) is a read/write control field that allows software to request suspension of the timer, while Bit 1 (HALT_ACK) is a read-only status field indicating whether the halt request has been acknowledged. In the current implementation, the timer enters the halted state only when both `dbg_mode` and `HALT_REQ` are asserted, as defined by `halt_active = dbg_mode & halt_req`. The `HALT_ACK` field directly reflects this active halt condition.

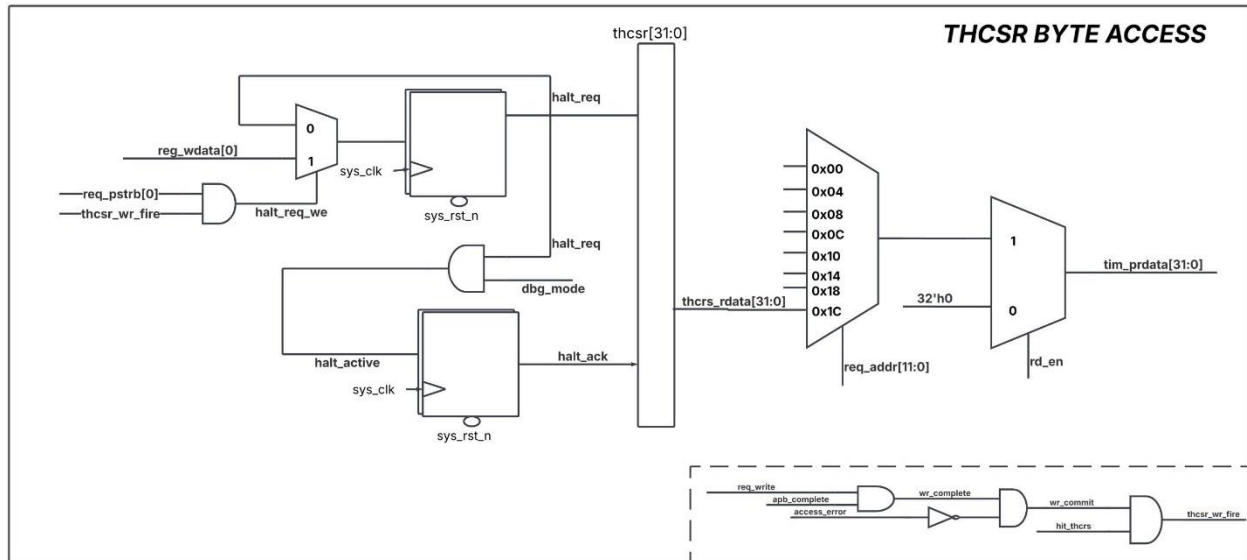


Figure 2-7 Logic diagram THCSR register

Table 2-3 THCSR register operation description

dbg_mode	halt_req	halt_ack	Counter
0	0	0	Run
0	1	0	Run
1	0	0	Run
1	1	1	Halt

Thus, asserting `HALT_REQ` alone does not stop the timer unless the system is also in debug mode. While halted, counter operation is suspended until the halt condition is released. Because `HALT_ACK` represents hardware-generated status, software writes must not modify or override this field; only `HALT_REQ` is software-programmable. This behavior is explicitly verified in the final checklist, including the read-only protection of `HALT_ACK`.

3. FUNCTIONAL DESCRIPTION

3.1. Timer Top Architecture

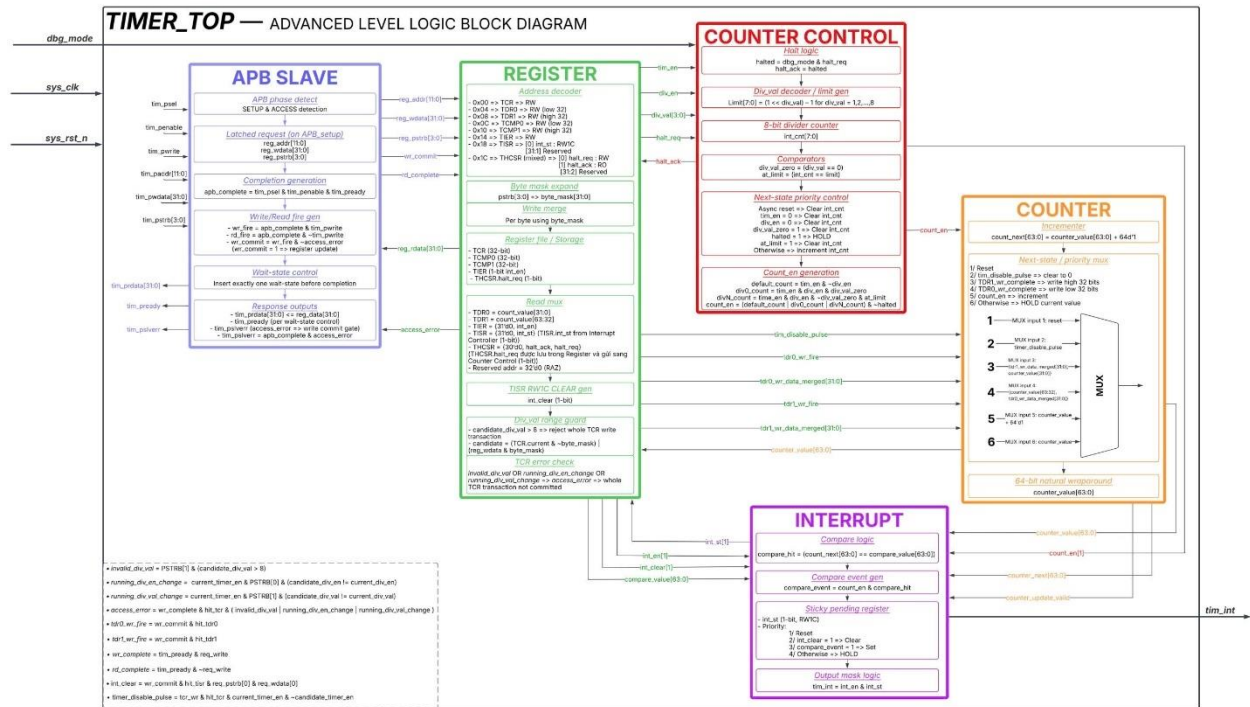


Figure 3-1 Advanced Level block diagram *TIMER_TOP*

The `TIMER_TOP` module is structured as a hierarchical architecture comprising five primary functional blocks: APB Slave, Register, Counter Control, Counter, and Interrupt. The advanced block diagram illustrates the interactions among these components while preserving a clear separation of responsibilities and state ownership.

The APB Slave block captures APB requests and manages wait-state control required for transaction completion. The Register block maintains the software-programmable configuration state, including timer_en, div_en, div_val, the timer compare registers, int_en, and halt_req. The Counter Control block manages the internal divider counter and generates the counting-enable condition based on the configured operating mode and debug-halt status. The Counter block exclusively owns the 64-bit timer state (count_value[63:0]), while the Interrupt block maintains the interrupt-pending state (int_st) and generates the external interrupt signal according to the pending status and interrupt-enable control.

This explicit ownership model eliminates multiple drivers and duplicated state across modules, thereby enhancing design readability, verification, debugging, and long-term maintainability. Consequently, the `TIMER_TOP` module primarily serves as an integration layer that interconnects these functional blocks and routes control, status, and data signals among them, rather than duplicating functional

logic internally. This architectural partitioning is consistent with the final implementation checklist, which mandates a one-owner-per-state organization for RTL design.

3.2. APB Slave

The APB Slave block provides the communication interface between the Timer IP and the external APB master. Its primary responsibilities include receiving and capturing APB requests, controlling transfer timing, and generating appropriate completion and error responses. In accordance with the APB protocol, each transaction proceeds through a SETUP phase followed by one or more ACCESS phases. For the advanced implementation, the Timer IP deliberately inserts exactly one wait state: PREADY remains low during the SETUP phase and the first ACCESS cycle, and is asserted in the second ACCESS cycle to complete the transfer.

The APB Slave further distinguishes protocol completion from register-state modification through the wr_complete, access_error, and wr_commit signals. Consequently, an APB write transaction may complete successfully at the protocol level while still being rejected at the functional level if it violates a Timer IP access rule. This organization ensures that illegal transactions return PSLVERR=1 at completion without causing unintended register updates, thereby improving robustness and design clarity.

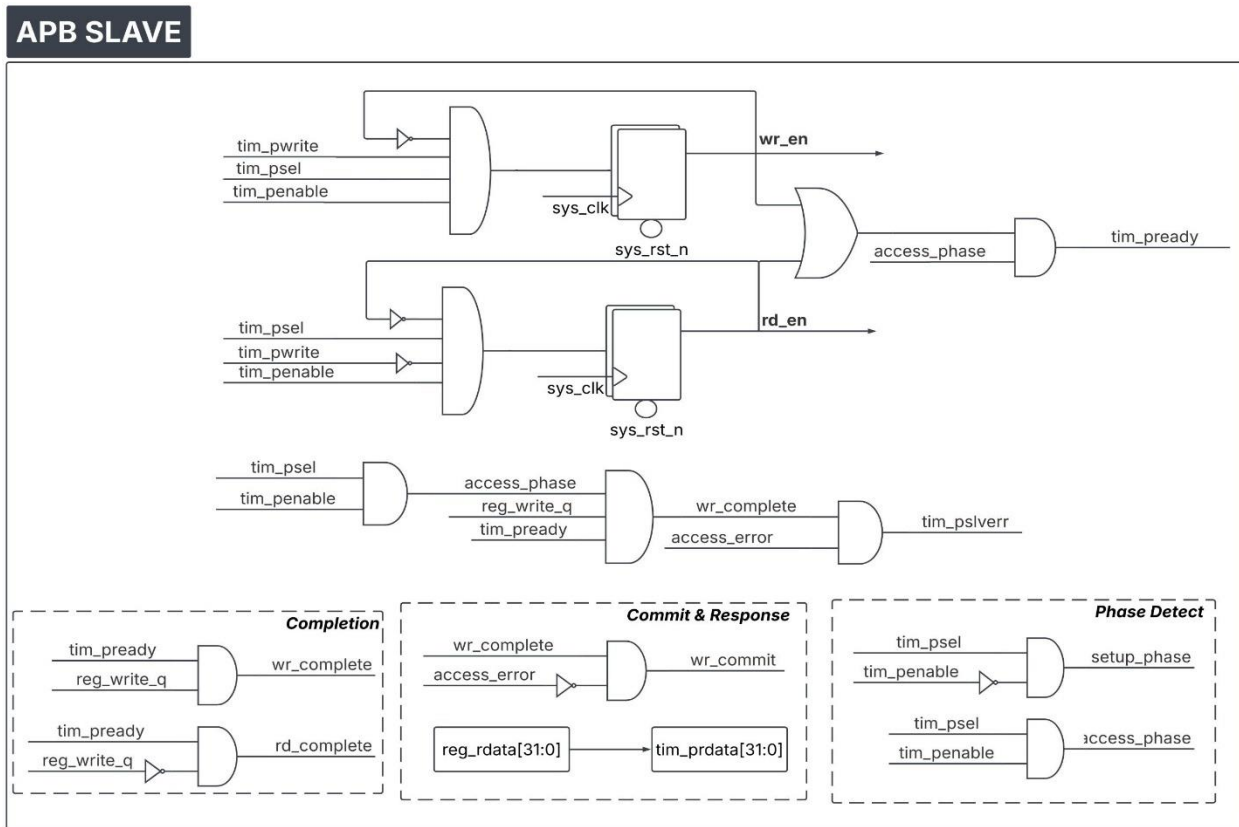


Figure 3-2 Logic diagram for Timer_apb_slave

3.2.1. SETUP & ACCESS phase

An APB transfer begins with the SETUP phase, which occurs when the slave is selected while PENABLE is low. During this phase, the transaction's address, transfer direction, write data, and write strobes are considered valid and are captured by the Timer IP. The subsequent cycle transitions into the ACCESS phase, during which PENABLE is asserted. In accordance with the APB protocol, the ACCESS phase may be extended by holding PREADY low, and the transfer is deemed complete only when PSEL, PENABLE, and PREADY are asserted simultaneously.

3.2.2. Request capture

To ensure transaction information remains stable throughout an extended ACCESS phase, the APB Slave captures the request during the SETUP phase. The captured information includes the APB address, write data, write strobe, and transfer direction, corresponding to internal request signals such as req_addr, req_wdata, req_pstrb, and req_write. These registered request values are subsequently utilized by the Register block for address decoding, byte-lane processing, legality checking, and read/write operation control. This approach prevents internal logic from relying directly on external bus signals that may change during a wait-state transaction, thereby ensuring robust and deterministic behavior.

3.2.3. One-cycle wait-state

In the advanced implementation, the Timer IP inserts exactly one APB wait state for every valid transaction. The expected response sequence is therefore: SETUP: PREADY = 0, ACCESS (first cycle): PREADY = 0, and ACCESS (second cycle): PREADY = 1. The first ACCESS cycle functions as the wait cycle, while the second ACCESS cycle completes the transaction. This behavior is formally defined in the final Verification Plan as requirement REQ-APB-002 and is validated by APB protocol-oriented testcases.

3.2.4. Read / write completion

A transaction is considered complete only during the final ACCESS cycle when PREADY is asserted. At this point, the APB Slave generates either wr_complete or rd_complete, depending on the captured transfer direction. These completion signals represent protocol-level events and are issued exactly once per transaction. They are therefore suitable for triggering subsequent register read or write operations without introducing repeated side effects during the wait-state cycle.

3.2.5. Write commit

For write transactions, protocol completion alone does not automatically authorize a register update. The design introduces a distinct wr_commit signal, conceptually defined as follows:

$$wr_commit = wr_complete \& \sim access_error$$

Here, wr_complete indicates that the APB write transaction has reached its completion cycle, while access_error signifies that the requested operation violates a Timer IP requirement. Consequently, wr_commit is asserted only when a write transaction is both completed and legal. This separation ensures that protocol handling and functional state modification remain clearly distinguished, thereby preventing unintended register updates and improving design robustness.

3.2.6. PSLVERR response

When a completed transaction violates an access rule, the Timer IP reports the error through PSLVERR. In such cases, the APB transaction still completes normally from the protocol perspective, with PREADY=1, while PSLVERR=1 indicates that the requested operation was rejected. At the same time, wr_commit remains deasserted, preventing any modification of the affected register state. Accordingly, an illegal transaction exhibits the following completion behavior: PREADY = 1 $\rightarrow$ PSLVERR = 1 $\rightarrow$ wr_commit = 0. This mechanism provides an atomic error response in which the transfer completes at the protocol level, but no partial or unintended state update is permitted.

3.2.7. Back-to-back transfers

The APB Slave also supports consecutive, or back-to-back, transactions. Once a transaction completes, the interface can immediately transition to the SETUP phase of the next transfer without requiring an additional idle cycle, provided that the APB master adheres to protocol timing requirements. For each new SETUP phase, the request information is recaptured, and the one-cycle wait-state mechanism is reinitialized for the new transaction. This ensures that each transfer is processed independently and that completion or error status from a preceding transaction does not inadvertently propagate into subsequent transfers.

3.3. Register Block

The Register Block provides the software-visible programming interface of the Timer IP and serves as the primary bridge between completed APB transactions and the internal functional blocks. It receives captured APB request information from the APB Slave, decodes the target register address, applies byte-level write masking according to PSTRB, constructs the candidate register value, and verifies the legality of the requested operation before permitting any state update. Only when a write transaction has successfully completed and passed all legality checks is wr_commit asserted. At that point, the Register Block either updates the corresponding configuration register or generates a command pulse for hardware-owned states such as the timer counter or interrupt status.

This organization ensures consistent implementation of register behavior, access permissions, byte-write semantics, and error handling across TCR, TDR, TCMP, TIER, TISR, and THCSR. By clearly separating protocol completion from functional state modification, the design enhances robustness, readability, and long-term maintainability.

3.3.1. Address decode

The address-decode logic determines which Timer IP register is selected based on the captured APB address. Each valid register address corresponds to a unique software-visible register, including TCR, TDR0, TDR1, TCMP0, TCMP1, TIER, TISR, and THCSR. The decoded selection signals are subsequently applied to both the read and write paths, ensuring that only the targeted register participates in the current transaction. Addresses outside the defined register map are treated as reserved and follow the specified RAZ/WI behavior, thereby maintaining protocol consistency and preventing unintended state modifications.

3.3.2. Register read mux

For read transactions, the Register Block employs an address-controlled read multiplexer to select the data returned to the APB Slave. Configuration registers are accessed from their stored software-visible state, while hardware-owned information—such as the current 64-bit counter value, interrupt-pending status, and halt acknowledgement—is provided through status inputs from the corresponding functional blocks. Reserved bits and reserved addresses consistently return zero, ensuring deterministic software readback and strict compliance with the defined RAZ/WI policy.

3.3.3. PSTRB byte merge

The advanced implementation supports byte-write access through PSTRB[3:0]. Each strobe bit controls one byte lane of the 32-bit write data, enabling software to update selected bytes while preserving the remaining contents of the register. Consequently, the Register Block does not apply the raw PWDATA value directly. Instead, it merges the selected write bytes with the existing register contents to construct a complete candidate value representing the state that would result if the transaction were accepted. This byte-merge mechanism is particularly critical for registers containing multiple fields distributed across different byte lanes, ensuring precise updates without unintended modifications.

3.3.4. RAZ/WI handling

Reserved register fields and undefined register addresses adhere to the Read-As-Zero/Write-Ignored (RAZ/WI) policy. A read operation targeting a reserved field or address consistently returns zero, while a write to such a location does not alter any implemented register state. For partially implemented registers, valid fields may still be updated according to PSTRB, whereas reserved bits remain unchanged internally and are always read back as zero. This behavior prevents software from inadvertently affecting undefined functionality and ensures a stable, deterministic register interface that facilitates future design extensions.

3.3.5. TCR candidate value & error detection

The Timer Control Register (TCR) requires additional legality checking because certain divider configurations are prohibited. This legality check is performed on the candidate TCR value after byte-lane merging, rather than directly on the raw write data. Such distinction is essential for correct PSTRB semantics. For example, if PSTRB[1] = 0, the byte containing DIV_VAL is not selected for modification; therefore, even if the corresponding raw PWDATA[11:8] encodes an illegal value such as 4'd15, that unselected data must not generate an error. The candidate-value approach ensures that only fields actually affected by the transaction participate in legality checking.

The TCR error logic evaluates the effective DIV_VAL, DIV_EN, and the current timer operating state. It detects prohibited divider values as well as illegal divider reconfiguration attempts while the timer is enabled. When such a violation is identified during a completed TCR write, access_error is asserted and the transaction is prevented from committing. This mechanism also satisfies the Verification Plan requirement that unselected illegal raw data must not produce a false APB error, thereby ensuring both correctness and protocol compliance.

3.3.6. Register command pulses

Not all software-visible registers are implemented as independent storage elements within the Register Block. For hardware-owned states, the Register Block instead generates one-cycle command pulses whenever a valid write is committed. Examples include write commands for TDR0 and TDR1, the timer-disable clear event, and the software-initiated clear command for the TISR INT_ST field. These pulses are forwarded to the corresponding Counter or Interrupt blocks, which remain the exclusive owners of their internal state.

This separation preserves the one-owner-per-state architectural principle, eliminates duplicate storage and multiple drivers, and ensures that software accesses and hardware updates are resolved through a clearly defined priority scheme. As a result, the design maintains both structural clarity and long-term maintainability.

3.3.7. Timer disable pulse

The `timer_disable_pulse` is a one-cycle control signal generated by the Register Block when a valid TCR write causes the timer-enable state to transition from 1 to 0. Its purpose is to distinguish the disable event from the disabled state itself. The pulse is produced only when the APB write has been successfully committed, the selected address corresponds to TCR, the current `timer_en` value is asserted, and the candidate TCR value requests `timer_en` to be cleared. Conceptually, the logic can be expressed as:

$$tcr_wr = wr_commit \& hit_tcr$$

$$timer_disable_pulse = tcr_wr \& timer_en \& \sim candidate_timer_en$$

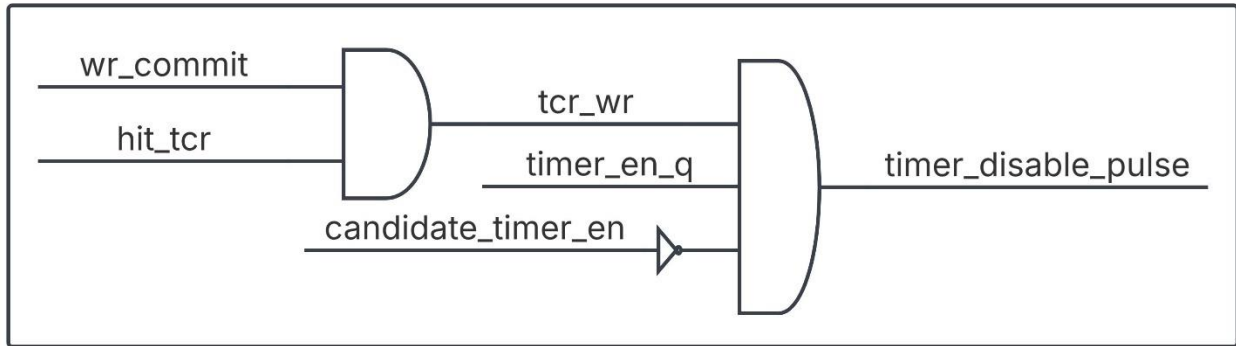


Figure 3-3 Logic diagram `timer_disable_pulse`

Because `wr_commit` is asserted only for a completed and legal APB write, an illegal TCR transaction cannot generate `timer_disable_pulse` or cause an unintended counter clear. When asserted, this pulse is forwarded to the Counter block, where it has higher update priority than a software TDR write or a normal counting event, and clears the entire 64-bit counter to zero. The pulse is not held active while the timer remains disabled; therefore, software can subsequently write TDR0 or TDR1 while `timer_en` = 0, and the programmed value is retained. This event-based implementation ensures that the counter is cleared

exactly once on a valid 1→0 transition of timer_en, rather than being continuously forced to zero throughout the disabled state.

3.3.8. Access error logic

The Access Error Logic validates TCR write transactions before any configuration state is modified. Because the Timer IP supports byte-write access through PSTRB, legality checks are performed on the candidate TCR value after byte-lane merging rather than directly on the raw APB write data. This ensures that only the byte lanes explicitly selected by the transaction participate in error evaluation. For instance, if the byte containing DIV_VAL is not selected, an illegal value present in the corresponding raw PWDATA bits must not generate a false access error.

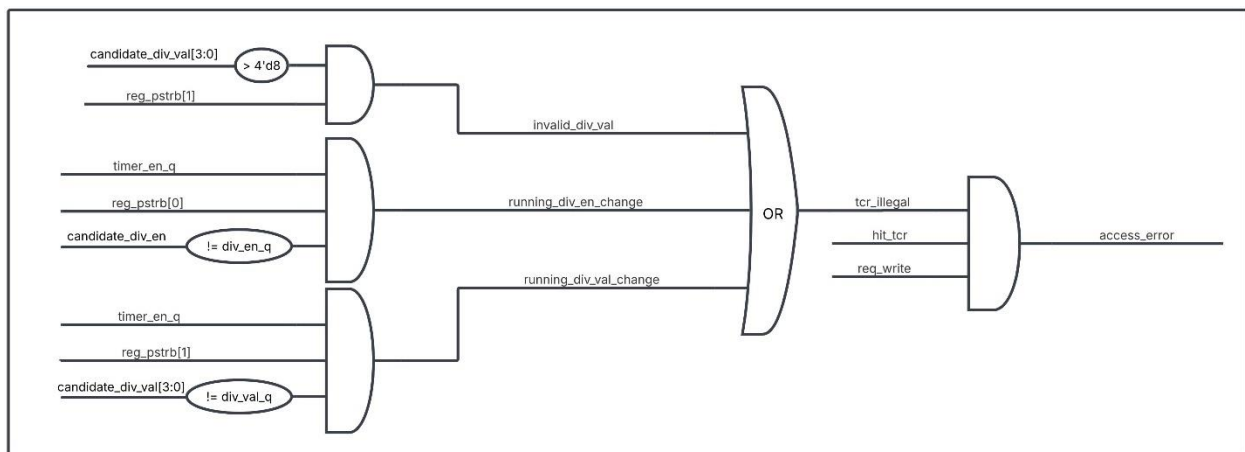


Figure 3-4 Logic diagram access_error

The logic evaluates three principal prohibited conditions: (1) an effective DIV_VAL greater than 8, (2) a change of DIV_EN while the timer is enabled, and (3) a change of DIV_VAL while the timer is enabled. The latter two checks compare the candidate configuration against the current stored values, ensuring that rewriting the same divider setting is not treated as a prohibited modification. These conditions are consolidated into a single TCR illegal-access indication.

When a completed write targets TCR and any prohibited condition is detected, access_error is asserted. The APB Slave then reports the failure by asserting PSLVERR during the final ACCESS cycle, while wr_commit remains deasserted so that no TCR field is updated. Consequently, the entire transaction is rejected atomically, and the previous register contents are preserved, preventing partial or unintended configuration changes.

3.4. Counter Control Block

The Counter Control block determines when the 64-bit timer counter is permitted to increment. Instead of generating a divided clock, this block derives a synchronous clock-enable signal (count_en) from the programmed timer configuration and the current divider state. All sequential elements within the Timer

IP continue to operate from the common sys_clk; count_en simply specifies whether the main counter performs an increment on a given rising edge. This approach preserves a single-clock-domain architecture and eliminates the timing and verification complexity typically associated with generated clocks.

3.4.1. Default counting mode

In the default counting mode, the timer is enabled while the programmable divider remains disabled (timer_en = 1, div_en = 0). Under this condition, div_val is ignored, and count_en is asserted on every active sys_clk cycle. As a result, the 64-bit timer counter increments once per system-clock cycle. This mode provides the highest counting rate and eliminates the need for the internal divider counter to participate in count scheduling.

3.4.2. Divider bypass mode

The divider bypass mode occurs when both the timer and divider are enabled, but div_val is programmed to zero (timer_en = 1, div_en = 1, div_val = 0). Although this configuration differs from the default mode at the register level, its functional counting behavior is identical: the main counter increments on every system-clock cycle. In this case, the divider path is effectively bypassed, and the internal divider counter remains held at its initial state.

3.4.3. Programmable divider mode

When timer_en = 1, div_en = 1, and div_val is programmed to a legal nonzero value between 1 and 8, the Counter Control block generates a periodic count_en pulse according to the selected counting interval. The effective period of the main counter is defined by 2^{div_val} active system-clock cycles. Consequently, larger divider values reduce the effective counting rate, while the entire design remains fully synchronized to sys_clk. The supported operating intervals are summarized below.

Table 3-1 Counter_control mode setting

DIV_VAL	Main Counter Period
0	1 clock
1	2 clocks
2	4 clocks
3	8 clocks
4	16 clocks
5	32 clocks
6	64 clocks
7	128 clocks
8	256 clocks

3.4.4. Internal divider counter

To implement programmable counting intervals without generating a derived clock, the Counter Control block maintains an internal 8-bit divider counter. This counter records the number of elapsed `sys_clk` cycles between successive increments of the main 64-bit counter. An 8-bit width is sufficient, as the largest supported counting interval is 256 cycles, requiring the internal state to represent values from 0 through 255. For each nonzero legal `DIV_VAL`, the terminal count is conceptually defined as:

$$limit = 2^{div_val} - 1$$

For example, when `DIV_VAL = 2`, the terminal value is 3, meaning the internal counter progresses through 0, 1, 2, 3 before permitting the main counter to increment. Once the terminal value is reached, the internal divider counter resets to zero, and a new counting period begins. This mechanism ensures precise scheduling of counter increments while preserving synchronization with `sys_clk`.

3.4.5. Count_en generation

3.4.5.1. First count_en generation

The `count_en` signal is derived from three mutually exclusive counting conditions: default counting, divider-bypass counting, and programmable divided counting. Conceptually, the count request can be expressed as:

$$default_count = timer_en \& \sim div_en$$

$$div0_count = timer_en \& div_en \& (div_val == 0)$$

$$divN_count = timer_en \& div_en \& (div_val \neq 0) \& (int_cnt == limit)$$

These three conditions are combined to determine when the main counter is eligible to increment. In the advanced implementation, the resulting count request is further masked by the debug-halt condition before producing the final `count_en` output. Thus, `count_en` functions strictly as a clock-enable control signal rather than a derived clock source. All counter state updates occur on the rising edge of `sys_clk`, with `count_en` determining whether an increment is performed on that edge.

COUNTER CONTROL

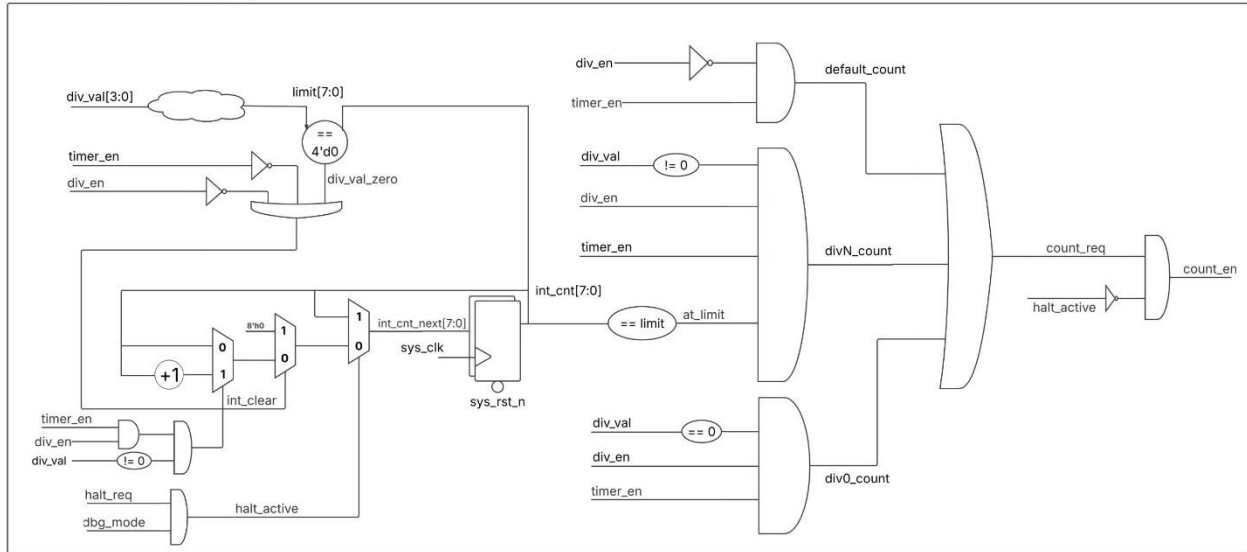


Figure 3-5 Logic diagram timer_counter_control

3.4.5.2. Second count_en generation

The original divN_count path was functionally valid, but its compound, state-correlated expression made full expression and FEC coverage unnecessarily difficult. To improve coverage observability and maintainability while preserving functionality, the RTL was refactored into three distinct components: bypass_count, count_request, and final global qualification. This restructuring aligns with the project's coverage methodology, which distinguishes genuine verification gaps from RTL constructs whose complexity hinders meaningful coverage.

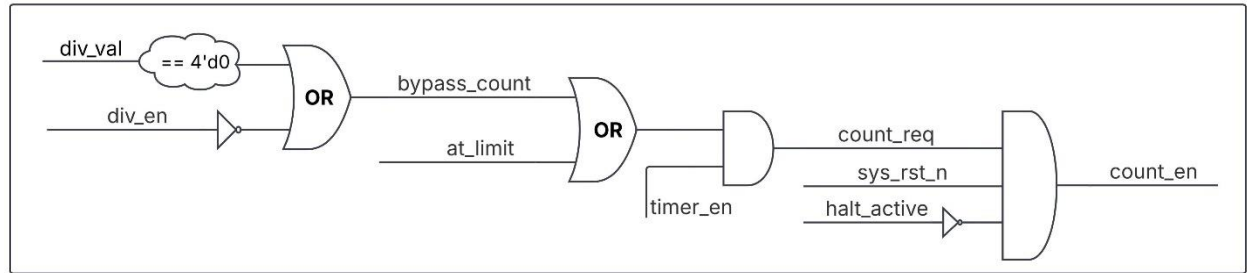


Figure 3-6 Logic diagram count_en (second version)

To simplify the logic while retaining the original behavior, the two clock-by-clock counting cases were unified into a single bypass condition:

$$\text{bypass_count} = \sim \text{div_en} | (\text{div_val} == 0)$$

This expression directly represents the two scenarios in which no internal divider delay is required. When div_en = 0, the timer operates in Default Mode and div_val is ignored. When div_en = 1 and div_val

= 0, the divider is enabled but functionally bypassed. In both cases, an enabled timer increments on every active system-clock cycle. For nonzero divider values, `bypass_count` evaluates to false, and the main counter increments only when the internal divider reaches its terminal state (`at_limit`).

The counting decision can therefore be expressed more compactly as:

$$\text{count_request} = \text{timer_en} \& (\text{bypass_count} | \text{at_limit})$$

and the final counter-enable signal is generated as:

$$\text{count_en} = \text{sys_rst_n} \& \sim \text{halt_active} \& \text{count_request}$$

This formulation separates the logic into three conceptual levels. First, `bypass_count` determines whether the current configuration requires immediate counting rather than divided counting. Second, `count_request` evaluates whether the timer configuration and divider phase currently request an increment. Finally, `count_en` applies global reset and debug-halt qualification, ensuring that no increment occurs while reset is asserted or while an effective debug halt is active.

The transformation preserves the original functional behavior. In Default Mode (`div_en = 0`), `bypass_count = 1`, so an enabled timer requests a count every cycle. In Divider Bypass Mode (`div_en = 1`, `div_val = 0`), `bypass_count = 1` again, producing the same one-count-per-clock behavior. For programmable divider values (1...8), `bypass_count = 0`, so `count_request` depends on `at_limit`; the main counter increments only when the internal divider reaches the terminal phase corresponding to the selected $2^{\text{div_val}}$ period. The internal divider state machine continues to provide phase sequencing and terminal detection independently of this combinational simplification.

The additional `sys_rst_n` and `~halt_active` qualification at the final `count_en` output establishes two global suppression conditions. When reset is active, `count_en` is forced low regardless of configuration. Likewise, when `halt_active = dbg_mode & halt_req` is asserted, any otherwise valid counting request is suppressed while the internal divider state is held, thereby preserving the divider phase for correct continuation after resume.

Overall, this refactored implementation improves coverage observability and simplifies the `count_en` expression without altering externally observable timer functionality or introducing a derived clock domain.

3.4.6. Debug halt and divider phase preservation

The Counter Control block also implements the debug-halt mechanism. A halt becomes active only when both the external `dbg_mode` input and the programmed `halt_req` field are asserted:

$$\text{halt_active} = \text{dbg_mode} \& \text{halt_req}$$

When `halt_active = 1`, the main counter must stop incrementing, and the internal divider counter must hold its current value rather than being cleared or allowed to continue running. Preserving the internal divider state is essential, as it maintains the partially completed counting interval across the halt period.

Once the halt condition is released, counting resumes from the saved divider phase rather than restarting a new interval.

For example, with `DIV_VAL = 2`, the main counter requires four active system-clock cycles per increment. If the timer is halted after part of this four-cycle interval has elapsed, the internal divider counter retains that partial progress during the halt and completes only the remaining cycles after resume. This guarantees that the number of active clock edges required for each timer increment remains consistent before and after debug suspension.

This implementation also preserves a single synchronous clock domain throughout the Timer IP: `sys_clk` drives all sequential logic, while `count_en` functions solely as an update-enable signal for the main counter. This design principle aligns with the project requirement that divider functionality be implemented without introducing an additional clock domain.

3.5. Counter

The Counter block is the exclusive owner of the 64-bit timer state (count_value[63:0]). Its primary responsibility is to maintain and update this state in response to four classes of events: asynchronous reset, a valid timer-disable transition, software writes to TDR0 or TDR1, and normal hardware counting through count_en. By consolidating the entire 64-bit counter state within a single block, the design eliminates duplicated storage and ensures deterministic behavior for carry propagation, software programming, overflow handling, and concurrent update scenarios.

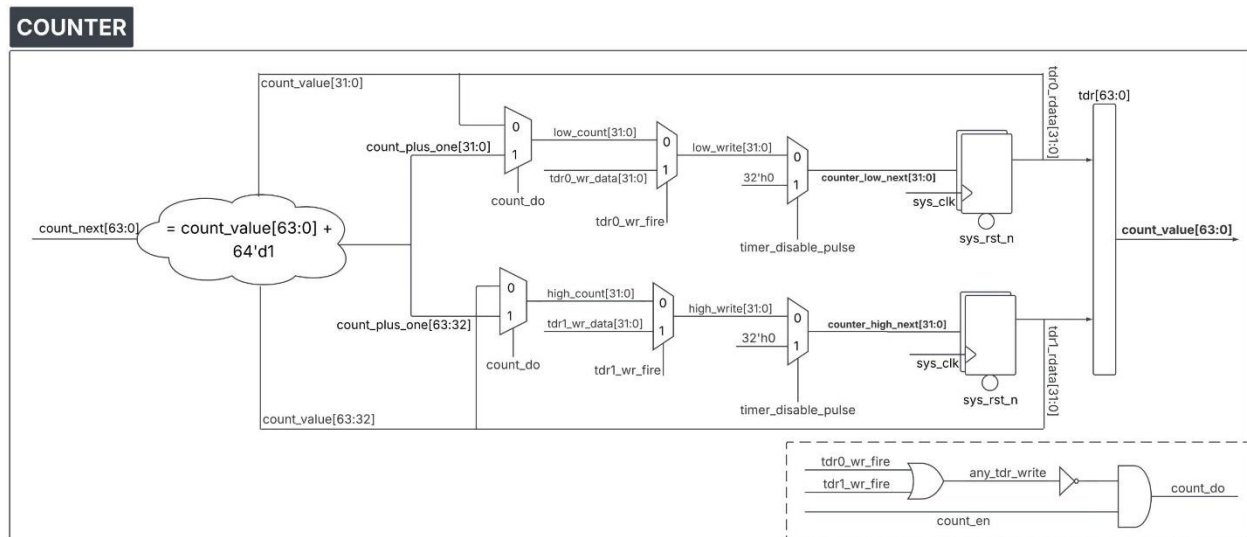


Figure 3-7 Logic diagram timer_counter

The counter update logic follows a clearly defined priority order: Asynchronous reset -> Timer disable pulse -> Software TDR write -> count en increment -> Hold

This ordering guarantees that reset always restores the counter to its initial state, a valid timer-disable transition clears the counter without permitting an additional increment, explicit software programming is not overwritten by a simultaneous hardware count, and normal counting occurs only when no higher-priority operation is active. The priority scheme is consistent with the verification baseline, which requires timer-disable events to override write/count activity and software TDR writes to take precedence over simultaneous hardware increments.

3.5.1. Counter increment

During normal operation, the Counter block increments the full 64-bit `count_value` by one whenever `count_en` is asserted and no higher-priority event is active. The increment is executed as a single 64-bit arithmetic operation rather than as two independent 32-bit additions, thereby ensuring that the counter functions as a continuous 64-bit count-up datapath. When `count_en` = 0 and no reset, disable, or software-write event occurs, the counter simply retains its current value. Consequently, in programmable divided modes, the counter remains unchanged between successive `count_en` pulses and advances exactly once for each valid counting event. This behavior is explicitly mandated by the counter requirements for both default and divided operation.

Conceptually, the normal counting operation can be expressed as:

```

if (count_en)
    count_value_next = count_value + 64'd1
else
    count_value_next = count_value

```

Importantly, `count_en` serves solely as a clock-enable condition rather than as a derived clock signal. The Counter block is clocked exclusively by `sys_clk`, ensuring that all sequential logic remains within a single synchronous clock domain.

3.5.2. Low-to_high carry

Although software interacts with the timer through the 32-bit TDR0 and TDR1 register views, the hardware counter operates as a unified 64-bit quantity. TDR0 maps to `count_value[31:0]`, while TDR1 maps to `count_value[63:32]`. Consequently, when the lower 32-bit word reaches 32'hFFFF_FFFF and the counter increments, the carry must propagate seamlessly into the upper 32-bit word. For example:

```

64'h0000_0000_FFFF_FFFF
+
64'h0000_0000_0000_0001
-----
64'h0000_0001_0000_0000

```

This result confirms that TDR0 and TDR1 are not independent counters, but rather software-visible portions of the same 64-bit state. The final verification plan explicitly validates this behavior by initializing

the counter to 0x00000000FFFFFFFF and confirming that a single hardware increment produces 0x0000000100000000.

3.5.3. 64-bit overflow and wrap

The Counter block implements natural unsigned 64-bit wrap-around behavior. When `count_value` reaches its maximum representable value: 64'hFFFF_FFFF_FFFF_FFFF, the next valid increment produces: 64'h0000_0000_0000_0000

The overflow condition does not halt the timer, trigger an implicit reset, or otherwise alter the timer configuration. Instead, the counter seamlessly continues normal operation from zero on subsequent `count_en` events. Executing the increment as a single 64-bit addition ensures both the bit-31-to-bit-32 carry and the full 64-bit wrap-around behavior occur naturally in the RTL. Project documentation explicitly identifies incorrect low-to-high carry handling as a critical implementation error to be avoided.

3.5.4. TDR software write

Software can directly initialize or modify the timer counter through writes to TDR0 and TDR1. A committed TDR0 write updates only the lower 32 bits while preserving the current upper half:

$$count_value_next = \{count_value[63:32], tdr0_wr_data\}$$

Similarly, a committed TDR1 write updates only the upper 32 bits while preserving the current lower half:

$$count_value_next = \{tdr1_wr_data, count_value[31:0]\}$$

The write data received by the Counter block has already been processed by the Register Block according to PSTRB, ensuring that unselected byte lanes retain their previous values. When a TDR software write coincides with a valid `count_en` counting event, the software write takes precedence and the hardware increment is suppressed for that cycle. This prevents an explicitly programmed value from being unintentionally altered by an additional +1 operation in the same edge. The final verification plan explicitly requires that a coincident TDR write produce the exact programmed result with no unintended increment.

3.5.5. Timer disable clear

The 64-bit counter is synchronously cleared when a successfully committed TCR transaction transitions `timer_en` from 1 to 0. This event is conveyed to the Counter block via the one-cycle `timer_disable_pulse` generated by the Register Block. When this pulse is asserted, the entire `count_value[63:0]` is reset to zero on the corresponding active clock edge.

The implementation deliberately responds to the transition event rather than continuously monitoring the disabled level. In other words, the Counter block must not be implemented as:

$$\begin{aligned} & \text{if } (!\text{timer_en}) \\ & \quad \text{count_value} \leq 0 \end{aligned}$$

Such logic would continuously force the counter to zero while the timer is disabled, preventing software from programming and retaining TDR values during the disabled state. Instead, only the committed 1→0 transition generates `timer_disable_pulse`. Once the timer is disabled and the pulse has returned low, software may write TDR0 or TDR1, and the written values remain stored until another valid update occurs. This behavior is mandated by the Golden-aligned implementation and verified in the final release checklist.

3.5.6. Counter update priority

Because multiple counter-update events may theoretically coincide on the same clock edge, the Counter block enforces an explicit priority scheme to ensure deterministic behavior.

Table 3-2 Counter-update events priority of `timer_counter`

Priority	Event	Counter action
1	Asynchronous reset	Clear <code>count_value</code> to 64'd0
2	<code>timer_disable_pulse</code>	Clear the complete 64-bit counter
3	TDR software write	Update the selected 32-bit half and preserve the other half
4	<code>count_en</code>	Increment the 64-bit counter by one
5	No active event	Hold the current counter value

The asynchronous reset holds the highest priority, as it establishes the defined initial state independently of all other activity. Next in priority is `timer_disable_pulse`, ensuring that a valid disable transaction immediately clears the counter and prevents any additional increment or software write from persisting on the disable edge. This ordering eliminates the common “extra count” issue, in which the counter might otherwise increment once more while `timer_en` is being cleared.

A valid TDR software write is prioritized over `count_en` because it represents an explicit software directive to establish a specific counter value. Allowing a simultaneous hardware increment to alter that value would create ambiguity and undermine determinism in the programming interface. Hardware increments are therefore executed only when no reset, disable, or software-write event has higher priority. If none of these events is active, the counter simply retains its previous value.

This priority ordering is explicitly reflected in the Counter logic diagram and is formally captured in the final verification requirements, which mandate disable-over-write/count and software-write-over-count behavior.

3.6. Interrupt

The Interrupt block is responsible for detecting the programmed timer-compare condition, maintaining the interrupt-pending state, and generating the external `tim_int` output. Its operation is intentionally structured into three logical stages: a full 64-bit comparison, a sticky pending-status register

(int_st), and an interrupt-enable mask (int_en). The complete interrupt path can therefore be represented as: “64-bit counter comparison -> int_st pending -> int_en masking -> tim_int”

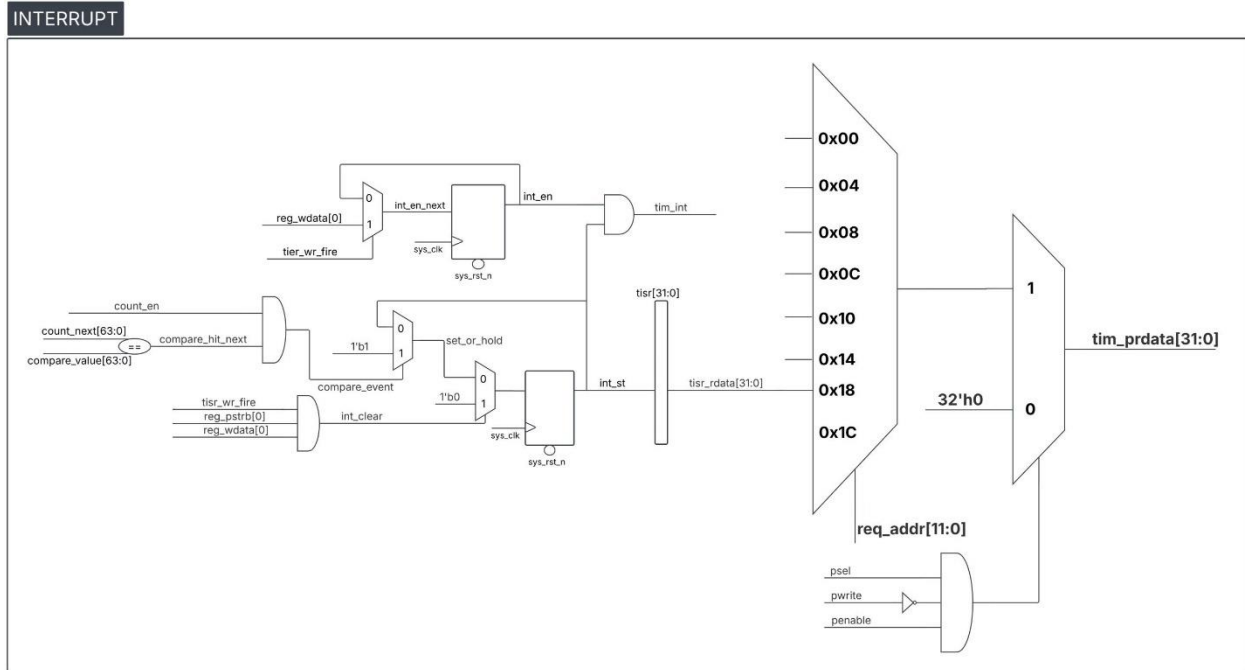


Figure 3-8 Logic diagram timer_interrupt

The Interrupt block exclusively owns the int_st state, while int_en and the 64-bit compare value are supplied by the Register Block. This separation ensures that interrupt detection, pending-state retention, software clearing, and output masking are implemented independently and deterministically. The external interrupt is generated according to:

$$tim_int = int_en \& int_st$$

Pending-state updates follow a defined priority order: asynchronous reset → software clear → full 64-bit compare match → hold. This ordering guarantees that software clearing takes precedence whenever multiple interrupt events occur on the same clock edge, thereby ensuring deterministic and predictable interrupt behavior.

3.6.1. 64-bit compare

The interrupt trigger condition is determined by comparing the complete 64-bit timer counter against the full 64-bit programmed compare value:

$$counter_match = count_value[63:0] == compare_value[63:0]$$

where:

$$compare_value[63:0] = \{TCMP1, TCMP0\}$$

The comparison must be performed across the entire 64-bit width rather than only the lower 32-bit word. Equality of `count_value[31:0]` and `compare_value[31:0]` alone is insufficient to generate a valid match if the upper 32-bit portions differ. This requirement ensures correct interrupt behavior throughout the full 64-bit timer range and is explicitly validated by verification scenarios in which the lower words are equal while the upper words are intentionally different.

3.6.2. Interrupt pending status

When a valid full-width compare condition is detected, the Interrupt block asserts the internal pending-status bit `int_st`. This state is implemented as a sticky hardware flag rather than as a direct combinational reflection of the comparator output. Once asserted, `int_st` remains active until explicitly cleared by software or reset by `sys_rst_n`. This sticky behavior ensures that the processor can reliably observe and service a timer event even after the immediate comparison condition has passed.

The `int_st` state is exposed to software through bit 0 of TISR, but ownership remains exclusively with the Interrupt block. The Register Block simply returns its current value during TISR readback and issues the software-clear command when required; it does not maintain a redundant copy of the pending state.

3.6.3. Interrupt masking

Interrupt pending status and interrupt output enable are treated as distinct functions. The `int_en` field in TIER does not determine whether a timer event may set `int_st`; instead, it solely governs whether an existing pending condition is propagated to the external `tim_int` output:

$$tim_int = int_en \& int_st$$

Consequently, if `int_st = 1` while `int_en = 0`, the pending interrupt remains recorded internally, but `tim_int` stays deasserted. If software subsequently sets `int_en = 1` while `int_st` is still asserted, `tim_int` becomes active without requiring a new compare event. Conversely, clearing `int_en` while an interrupt is pending suppresses only the external output and does not clear `int_st`. This mechanism ensures the required hardware-based, maskable, level-sensitive interrupt behavior.

3.6.4. RW1C clear

The interrupt pending state is cleared by software through the Read/Write-1-to-Clear (RW1C) behavior of TISR bit 0. A valid clear request is generated only when a committed APB write targets TISR, byte lane 0 is selected by `PSTRB[0]`, and the corresponding write-data bit is asserted: “`PSTRB[0] = 1` and `PWDATA[0] = 1`”.

Writing 0 to `INT_ST` has no clearing effect, and writes to unselected or reserved byte lanes are ignored. The Register Block translates a valid RW1C transaction into the one-cycle `int_clear` command, which is then forwarded to the Interrupt block. This organization confines APB decoding and byte-access handling to the Register Block, while leaving ownership of the actual pending-state update exclusively with the Interrupt block.

3.6.5. Clear / Match priority

Because a software-clear request and a hardware compare condition may occur on the same active clock edge, the Interrupt block defines an explicit update-priority scheme: Asynchronous reset -> int_clear -> counter_match -> Hold previous int_st

The asynchronous reset has the highest priority and immediately initializes the pending state to zero. During normal operation, int_clear takes precedence over counter_match. Thus, if software clears TISR in the same cycle that the 64-bit counter matches the programmed compare value, the resulting int_st state is cleared rather than set. If neither reset nor clear is active, a valid full-width match asserts the pending state; otherwise, the previous value is retained. This priority ordering ensures deterministic handling of concurrent hardware and software events.

3.6.6. Interrupt reassertion

Although a software-clear operation has priority over a compare match within the cycle in which both occur, clearing int_st does not modify either the timer counter or the programmed compare value. Consequently, if the full 64-bit equality condition remains valid on the subsequent clock cycle, the hardware may reassert int_st in accordance with the defined comparison behavior.

This reassertion mechanism is particularly relevant when the timer counter remains unchanged across multiple system-clock cycles, such as during divided counting, or when software writes TDR so that the resulting counter value equals the programmed compare value. In these cases, software may successfully clear the pending state for one cycle, but the Interrupt block can reassert it in a later cycle while the compare condition remains true. The distinction between clear-priority within a single cycle and re-evaluation of the match condition across subsequent cycles is an essential aspect of the final interrupt behavior.

3.6.7. Counter operation after interrupt

An interrupt event does not halt or reset the timer counter. Once int_st is asserted—regardless of whether tim_int is currently enabled or masked—the Counter and Counter Control blocks continue to operate according to the programmed timer configuration. Normal counting, divided-mode operation, carry propagation, and 64-bit overflow therefore proceed independently of the interrupt-pending state.

This separation is essential because int_st serves solely as a record of a timer event and must not be interpreted as a counter-enable or halt condition. Similarly, tim_int functions only as the externally visible interrupt indication and has no direct influence on timer progression. As a result, the timer continues to generate subsequent count values and naturally wraps after overflow, even while a previous interrupt remains pending until explicitly cleared by software.

3.7. Debug Halt Operation

The Debug Halt function enables the Timer IP to temporarily suspend timer progression while the system operates in debug mode. A halt condition becomes active only when both the external dbg_mode

input and the software-programmable `halt_req` field in THCSR are asserted. The active halt condition and its acknowledgement are therefore defined as:

$$\text{halt_active} = \text{dbg_mode} \ \& \ \text{halt_req}$$

$$\text{halt_ack} = \text{halt_active}$$

When `halt_active` = 1, the Timer IP suppresses the final counter-enable signal so that `count_en` = 0. As a result, both the 64-bit main counter and the internal divider counter hold their current values. The internal divider state must be preserved rather than reset, as it represents the partially completed counting phase of the programmable divider. This phase-preservation requirement ensures that the timer resumes precisely from the point at which it was halted, thereby maintaining the correct number of active system-clock cycles for each counter increment.

3.7.1. Counter operation after interrupt

The halt operation is initiated through the `halt_req` field of the Timer Halt Control and Status Register (THCSR). Although `halt_req` is programmable by software, asserting this bit alone does not suspend the timer. The request becomes effective only when the external `dbg_mode` signal is simultaneously asserted. Thus, if `halt_req` = 1 while `dbg_mode` = 0, the timer continues normal operation and `halt_ack` remains deasserted. This gating mechanism prevents an ordinary software write from unintentionally halting the timer outside the intended debug environment.

3.7.2. Counter operation after interrupt

The `halt_ack` field is a hardware-generated, read-only status indicator that directly reflects the effective halt condition. In the current implementation:

$$\text{halt_ack} = \text{halt_active}$$

Accordingly, `halt_ack` is asserted only when both `dbg_mode` and `halt_req` are high. When asserted, it confirms that timer progression has been suspended and that both the main counter and the internal divider phase are being held. Because `halt_ack` is hardware-controlled, software writes to this field are ignored and must not alter its value.

3.7.3. Counter operation after interrupt

When a halt condition is entered, the main counter ceases incrementing and the internal divider counter neither advances nor resets. Instead, both values remain unchanged for the entire duration of the halt. This behavior is critical in programmable divider mode, as the internal divider counter records the number of active clock cycles already elapsed within the current counting period.

For example, assume the timer is operating in divide-by-8 mode and the internal divider counter has reached phase 5 when the halt condition becomes active. During the halt interval, the divider state remains at 5 regardless of how many `sys_clk` cycles occur. After the halt condition is released, the internal divider resumes from the preserved phase rather than restarting from zero: “Before halt : `int_cnt` = 5; During halt : `int_cnt` = 5; After resume: `int_cnt` = 6 → 7 → terminal count”.

At the terminal phase, the next valid counting event is generated and the main counter increments as expected. Resetting the internal divider counter to zero during halt would discard the already completed portion of the divider period, thereby extending the first counting interval after resume. Conversely, allowing the divider to continue running during halt would shorten the remaining interval. The correct behavior is therefore to hold the divider phase during halt and resume from the preserved state. This requirement is explicitly validated in the final verification plan through dedicated halt-phase freeze and resume checks, including HLT-004 and HLT-005.

3.8. Reset Behavior

The Timer IP employs an active-low asynchronous reset (`sys_rst_n`) to initialize all functional state elements to their defined reset values. Reset extends beyond register initialization; it also establishes a known state for the 64-bit counter, internal divider phase, interrupt status, debug-halt control, and APB transaction-control logic. Because the reset is asynchronous, asserting `sys_rst_n = 0` immediately forces the affected sequential state into its reset condition without waiting for a rising edge of `sys_clk`. Once reset is released, normal synchronous operation resumes on subsequent rising clock edges.

The principal reset values of the Timer IP are summarized below:

Table 3-3 Reset value of timer IP

State	Reset Value
TCR	32'h0000_0100
Counter / TDR0 / TDR1	64'h0000_0000_0000_0000
TCMP0 / TCMP1	64'hFFFF_FFFF_FFFF_FFFF
TIER	32'h0000_0000
TISR / int_st	32'h0000_0000
THCSR.halt_req	1'b0
Internal divider counter	8'h00
APB request / wait-state control	Cleared

The TCR reset value of 32'h0000_0100 corresponds to `timer_en = 0`, `div_en = 0`, and `div_val = 1`. Although `div_val` is initialized to 1, the timer remains disabled after reset and the divider is inactive because both `timer_en` and `div_en` are cleared. The 64-bit timer counter is reset to zero, while the combined compare value is initialized to 64'hFFFF_FFFF_FFFF_FFFF. Both the interrupt-enable and interrupt-pending states are cleared, ensuring that `tim_int` remains deasserted after reset. The debug-halt request is also cleared, and the internal divider counter is returned to its initial phase.

Reset additionally clears the APB Slave transaction state, including any captured request information and wait-state tracking. If reset is asserted during an APB transfer, the partially processed

transaction is abandoned and no register write is committed from that interrupted transfer. After reset release, the APB Slave restarts from its idle/initial transaction state and waits for a new valid SETUP phase. This mechanism prevents stale request information or incomplete wait-state tracking from propagating into subsequent transactions.

Reset has the highest priority over all normal Timer IP activities. If reset is asserted while the counter is incrementing, the counter is cleared rather than completing the increment. If reset occurs during a TDR software write, the reset value overrides the programmed data. Similarly, reset clears `int_st` even if a compare match or software interrupt-clear event is present, resets the internal divider state even if the timer is halted, and clears the debug-halt control state regardless of the current `dbg_mode`. Consequently, reset provides a deterministic recovery mechanism from any operating state of the Timer IP.

This reset priority is particularly critical for verification, as the Final Project requires reset behavior to be validated not only from an idle condition but also while the design is actively performing APB transactions, counting, servicing an interrupt, or operating in debug-halt mode. In all such scenarios, assertion of `sys_rst_n` must override ongoing activity and restore the Timer IP to its specified initial state without leaving residual transaction, counter, interrupt, or divider state after reset release.

4. VERIFICATION SUMMARY

Verification of the Timer IP was executed as an integral element of the overall development process and was derived directly from the Advanced Level Final Project requirements. The verification flow was structured around the Verification Plan (VPLAN), reusable APB testbench infrastructure, automated checkers, directed testcases, comprehensive regression, and code-coverage analysis. Its objective was not only to confirm successful RTL compilation and simulation, but also to demonstrate that the implemented functionality satisfied all required specifications, including APB protocol compliance, register semantics, timer counting modes, 64-bit counter operation, interrupt mechanisms, debug-halt functionality, error handling, and defined priority rules.

The final VPLAN establishes explicit requirement-to-testcase traceability across reset behavior, APB transactions, register access, TCR error handling, counter operation, debug halt, interrupt functionality, system integration, and coverage closure. Verification was therefore embedded as a core component of the Final Project flow rather than treated as a separate post-design activity. In alignment with the course framework, the process was systematically organized through planning, testbench and checker development, testcase execution, coverage analysis, and release preparation—ensuring rigorous and deterministic validation of the Timer IP.

4.1. Verification Environment

The verification environment is structured into distinct simulation, testbench, and directed-testcase components, providing a reusable and maintainable framework for validation.

```

RTL
├─ timer_top.v
├─ timer_apb_slave.v
├─ timer_reg.v
└─ ...

SIM
├─ Makefile
├─ pat.list
├─ run.csh
└─ ...

TB
├─ test_bench.v
├─ apb_tasks.vh
├─ checker_tasks.vh
└─ timer_define.vh

TESTCASES
└─ 15 directed testcases
  
```

Figure 4-1 Timer_top structure

The SIM directory contains the simulation control infrastructure, including the Makefile, regression pattern list, execution scripts, and generated outputs. The TB directory hosts the common testbench components: `test_bench.v` defines the top-level verification environment, `apb_tasks.vh` implements reusable APB read/write transactions, `checker_tasks.vh` provides automated expected-versus-actual checking with PASS/FAIL reporting, and `timer_define.vh` maintains shared constants and register definitions. The final checklist explicitly requires reusable APB tasks supporting wait-state, error-response, and PSTRB behavior, together with automated checking to ensure correctness beyond waveform inspection.

Each directed testcase is designed to run independently, beginning from a defined reset and setup condition so that results are not dependent on execution order. This independence allows testcases to be executed individually during debugging or collectively as part of full regression. The testbench drives APB transactions according to the required SETUP and ACCESS phases, with completion observed only when the slave asserts PREADY. Functional correctness is evaluated through automated checkers, while waveform inspection is reserved for detailed timing analysis and debugging.

4.2. Verification Strategy

The verification strategy integrates requirement-based directed testing, corner-case analysis, integration testing, and coverage-driven closure. Each design requirement is explicitly represented in the Verification Plan (VPLAN) by a unique identifier, verification objective, precondition, stimulus sequence, and a clearly defined PASS criterion. This structured approach ensures full traceability between the design specification and the corresponding verification evidence.

Verification begins with fundamental behaviors such as reset handling, basic register access, and end-to-end timer operation. More advanced scenarios then address APB wait-state timing, back-to-back transfers, PSTRB byte access, RAZ/WI behavior, illegal TCR transactions, 64-bit carry and overflow, programmable divider timing, debug-halt phase preservation, interrupt masking, RWIC clearing, and concurrent-event priority resolution.

Special emphasis is placed on boundary and interaction conditions. For instance, an illegal TCR transaction must complete with an APB error response while producing no register or counter side effects. A write to an unselected TCR byte containing an invalid DIV_VAL must not generate a false error. Likewise, a software write to TDR coincident with a hardware increment must preserve the programmed value, and a valid timer_en transition from 1→0 must clear the counter without allowing an additional increment. These scenarios validate not only individual functional correctness but also the integrity of inter-block ownership and priority rules.

4.3. Testcases Summary

The final verification regression comprised 15 directed testcases, all of which completed with PASS status in accordance with the VPLAN testcase matrix and checklist.

Table 4-1 Total testcases

#	Testcase	Main Purpose	Status
1	tc_reset	Basic reset behavior	PASS
2	tc_smoke_1pass	End-to-end timer smoke scenario	PASS
3	tc_register	Register map and attribute validation	PASS
4	tc_register_byte_access	PSTRB byte access and byte-qualified RW1C	PASS
5	tc_interrupt_check	Golden-aligned interrupt behavior	PASS
6	tc_apb_protocol_check	APB SETUP/ACCESS and one-wait-state protocol	PASS
7	tc_apb	APB integration and back-to-back transactions	PASS
8	tc_counter_64bit	64-bit carry, wrap-around, and TDR write behavior	PASS
9	tc_counter_mode	Default, DIV0, and DIV1–DIV8 counting modes	PASS
10	tc_halt_debug	Debug halt and divider-phase preservation	PASS
11	tc_interrupt_corner	RW1C, full-64-bit compare, priority, and reassertion	PASS
12	tc_reserved_robustness	Reserved address/field RAZ/WI behavior	PASS
13	tc_reset_corner	Reset assertion during active operation	PASS
14	tc_tcr_error_atomic	Illegal TCR access and atomic rejection	PASS
15	tc_coverage_closure	Directed residual coverage closure	PASS

The testcase suite was deliberately expanded beyond basic smoke and register validation to ensure comprehensive functional and corner-case coverage. For example, tc_counter_64bit validates low-to-high carry propagation, full-width wrap-around, software counter programming, and software-write priority. tc_counter_mode checks Default Mode, DIV0 bypass, and all legal programmable divider values from DIV_VAL = 1 through 8, including the required 2^{div_val} counting periods. “tc_halt_debug” confirms that both the main counter and internal divider counter hold their values during halt and resume from the preserved divider phase. Finally, tc_interrupt_corner verifies full 64-bit equality, RW1C clear behavior, clear-over-match priority, interrupt reassertion, and additional Golden-aligned corner conditions.

4.4. Key Verification Scenarios

Critical verification scenarios were emphasized to address requirements vulnerable to subtle implementation errors. Representative simulation waveforms provide timing-level evidence of key behaviors, including APB wait-state handling, error responses without unintended state changes, byte-write operations, 64-bit counter functionality, programmable modes, interrupt handling, debug halt, timer-disable clearing, and asynchronous reset. These waveform observations complement automated checkers and regression results by illustrating the temporal alignment of control signals, internal states, and external outputs.

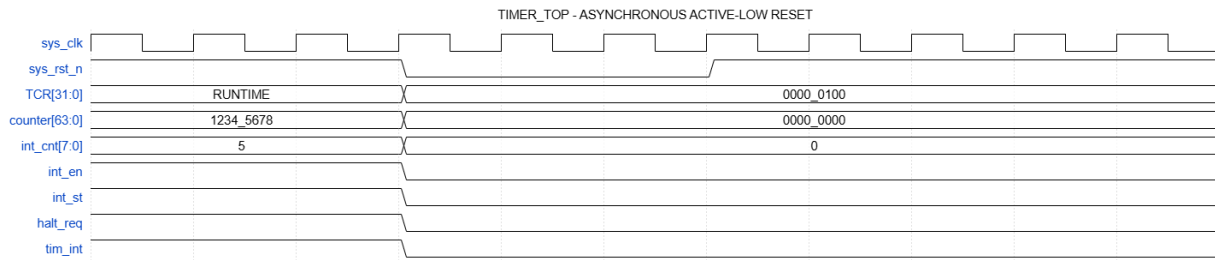


Figure 4-2 Timer_top - Asynchronous active-low reset waveform

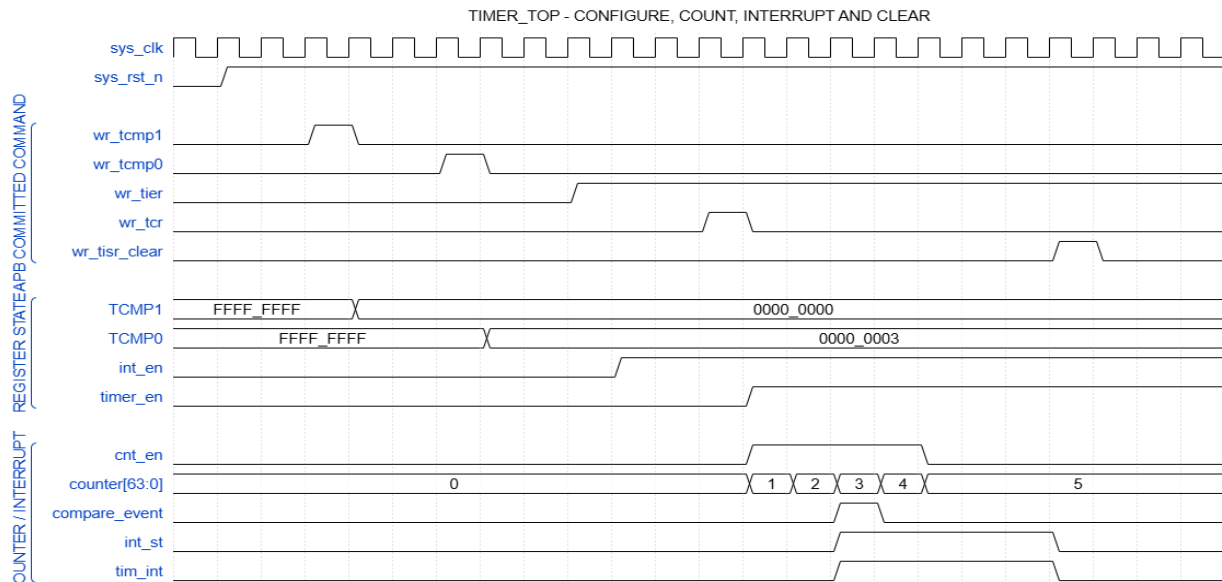


Figure 4-3 Timer_top - Configure, count, interrupt and clear waveform

For the APB interface, verification ensures proper request capture during the SETUP phase, enforcement of exactly one wait state, completion only in the second ACCESS cycle, correct PREADY and PSLVERR timing, and support for back-to-back transfers. Register state updates are confirmed to occur only after final ACCESS completion, while illegal transactions must not produce partial or unintended state changes.

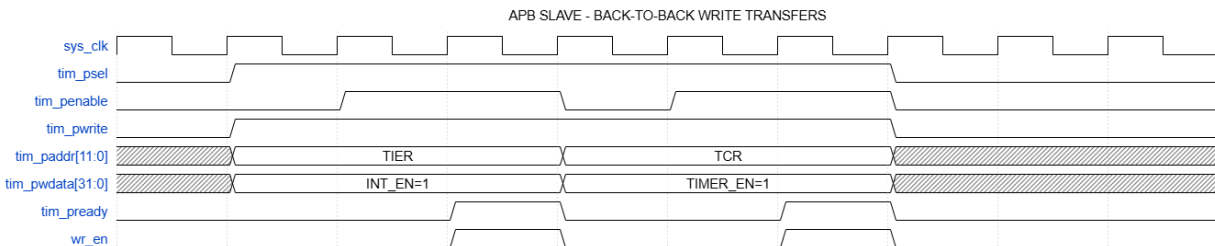


Figure 4-4 APB Slave - Back-to-back write transfers waveform

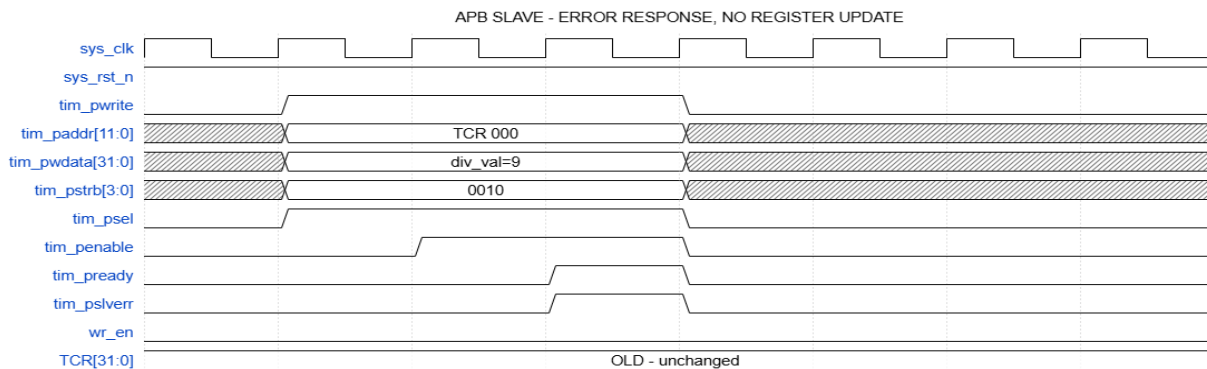


Figure 4-5 APB Slave - Error response & no register update waveform

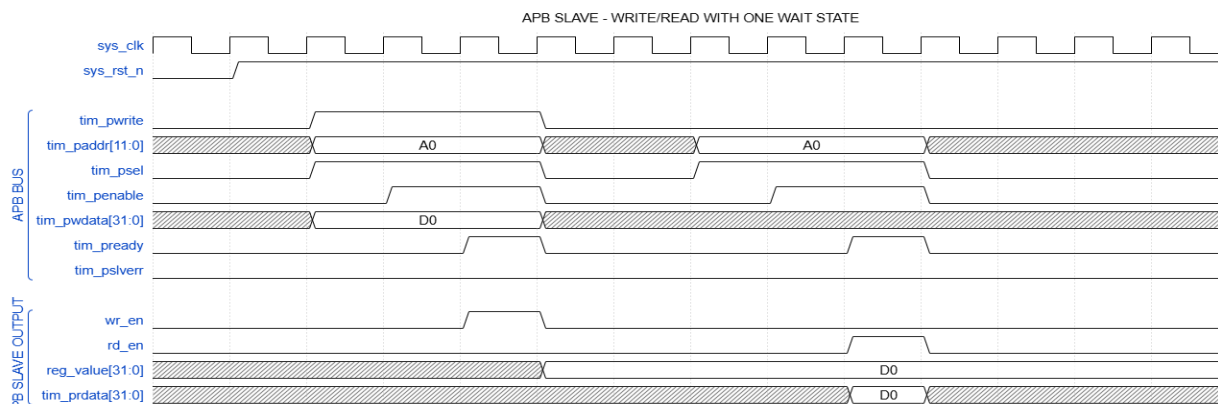


Figure 4-6 APB Slave - Write/Read with one wait state waveform

For register and byte-access behavior, each PSTRB lane is checked to confirm that only the selected byte is modified. Reserved bits and addresses are verified to exhibit RAZ/WI behavior. Clearing TISR is validated to occur exclusively when byte lane 0 is selected and PWDATA[0] = 1. Legality checking of TCR values is performed on the merged candidate value, ensuring that invalid raw data in an unselected byte does not trigger a false error.

REGISTER - ADDRESS DECODE AND READ MUX

req_addr[11:0]	000	004	008	00C	010	014	018	01C	020
register selected	TCR	TDR0	TDR1	TCMP0	TCMP1	TIER	TISR	THCSR	RESERVED
reg_rdata[31:0]	TCR_VALUE	CNT[31:0]	CNT[63:32]	TCMP0	TCMP1	INT_EN	INT_ST	HALT_ACK/REQ	0000_0000

Figure 4-7 Register - address decode and read mux waveform

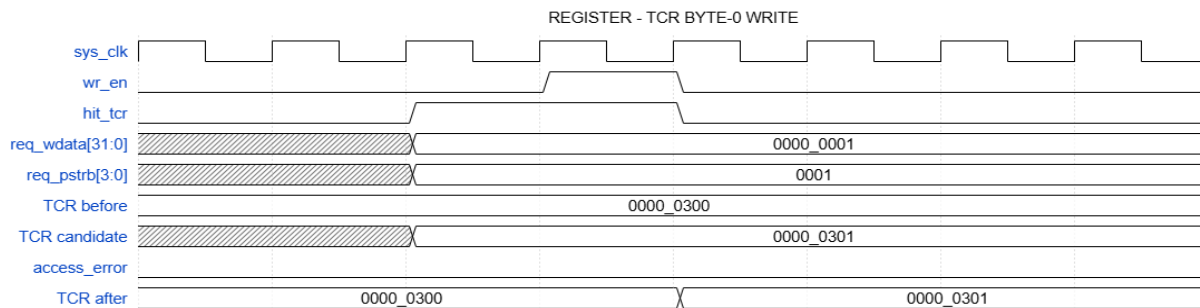


Figure 4-8 Register - TCR byte0 write waveform

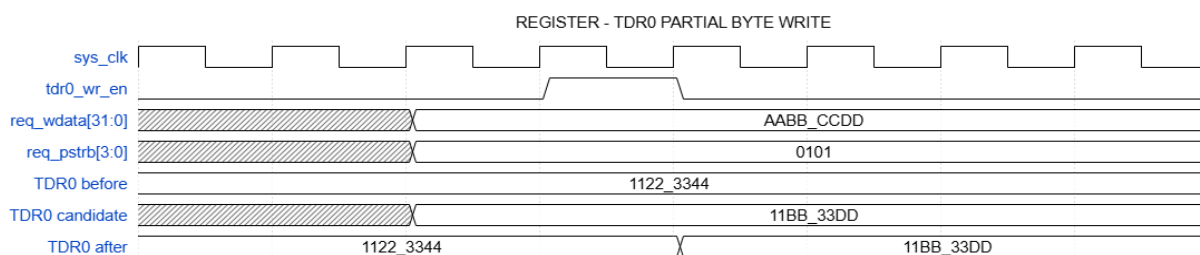


Figure 4-9 Register - TDR0 byte write waveform

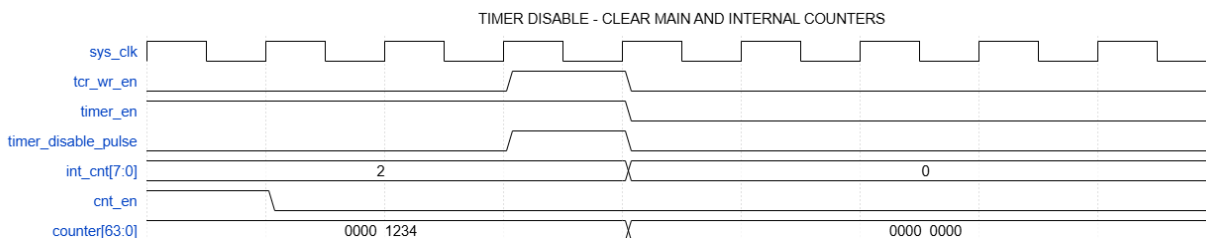


Figure 4-10 Register - Timer_disable_pulse waveform

For the counter, verification covers full 64-bit increment, low-to-high carry propagation, natural wrap-around, software writes to TDR, software-write-over-count priority, and counter clearing only on a valid timer_en 1→0 transition. Default Mode is validated to increment on every sys_clk, DIV0 bypass is confirmed to yield the same effective period, and programmable divider values from 1 through 8 are checked against the required $2^{\text{div_val}}$ counting intervals.

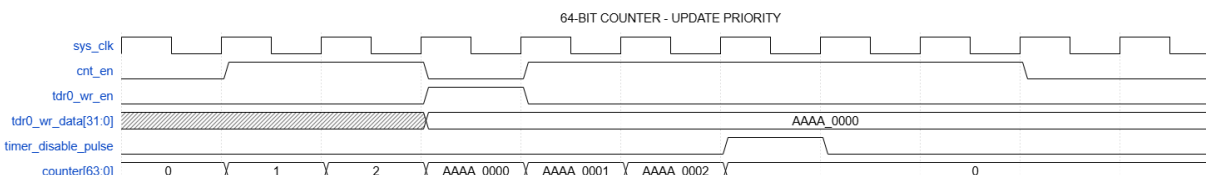


Figure 4-11 Counter - update priority waveform

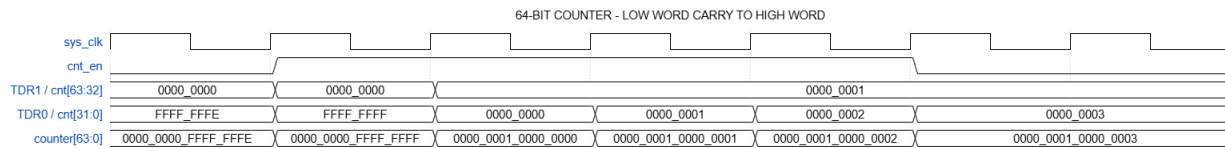


Figure 4-12 Counter - Low-to-high waveform

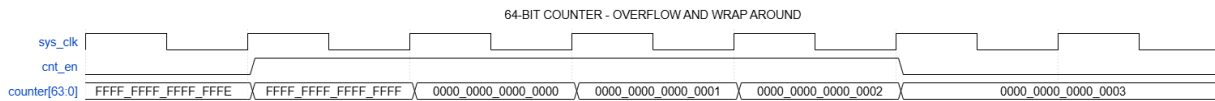


Figure 4-13 Counter - Overflow & wrap around waveform

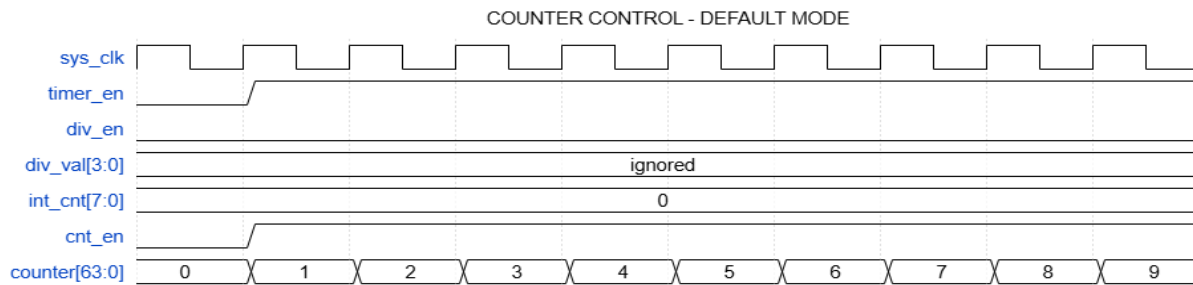


Figure 4-14 Counter control - Default mode waveform

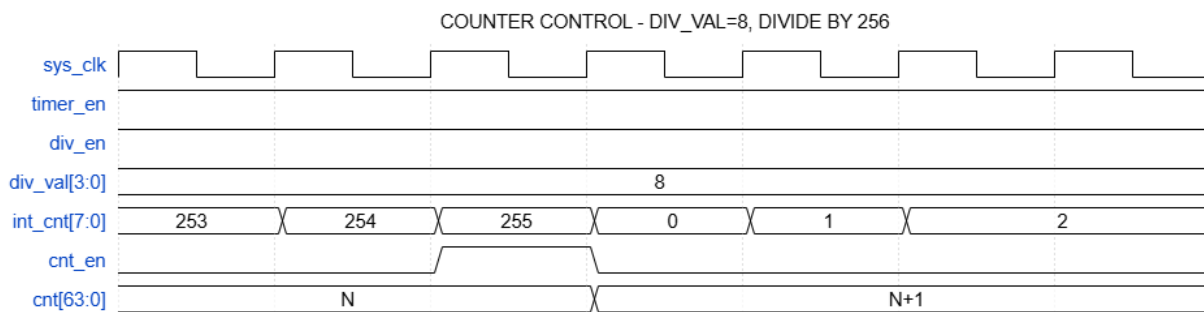


Figure 4-15 Counter control - case div_val = 8 waveform

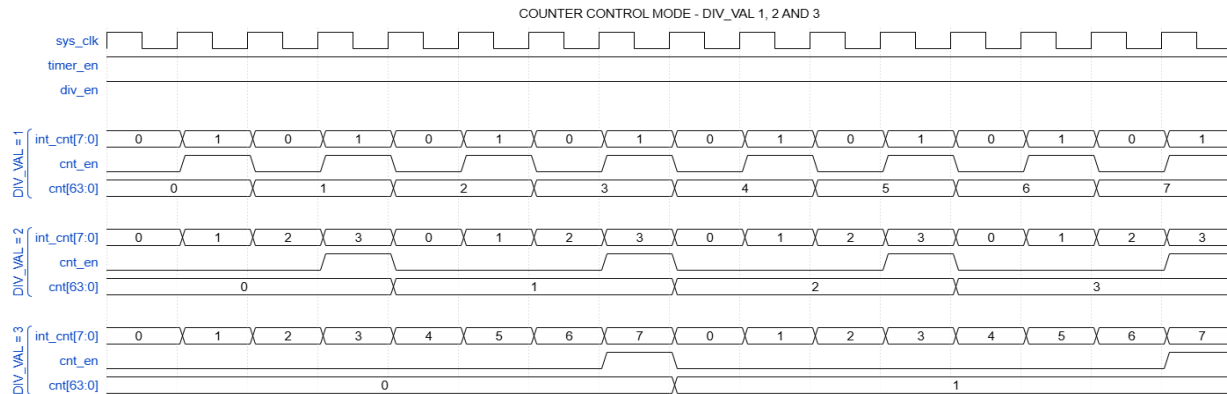


Figure 4-16 Counter control - case div_val = 1,2,3 waveform

For debug halt, verification confirms that halt_req alone does not stop the timer when dbg_mode = 0, while dbg_mode & halt_req correctly forces count_en = 0 and halt_ack = 1. Most importantly, the internal divider phase must hold during halt and resume from the preserved value without introducing an extra or missing count.

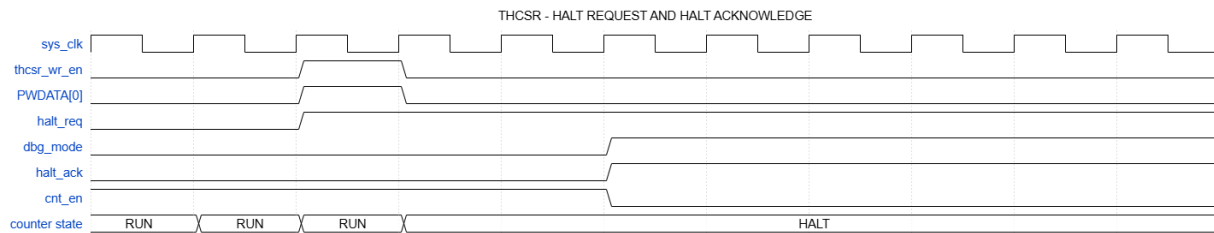


Figure 4-17 Halt_req & halt_ack waveform

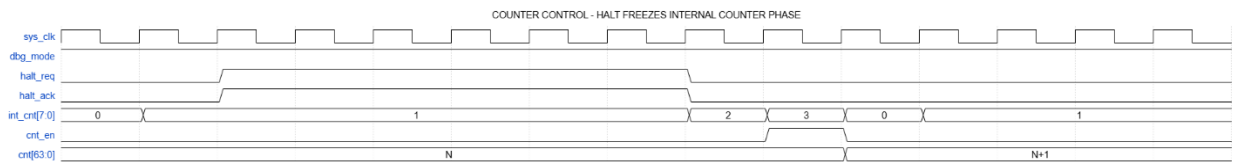


Figure 4-18 Counter control - halt freeze waveform

For the interrupt mechanism, the comparator is validated across the complete 64-bit timer and compare values. A dedicated testcase ensures that equality of the lower 32-bit words with differing upper words does not generate a pending interrupt until full 64-bit equality is achieved. Verification also confirms that int_en masks only tim_int, that TISR employs byte-qualified RW1C behavior, that int_clear has priority over a same-cycle compare match, that pending status may reassert if equality persists, and that interrupts do not interfere with normal counter progression.

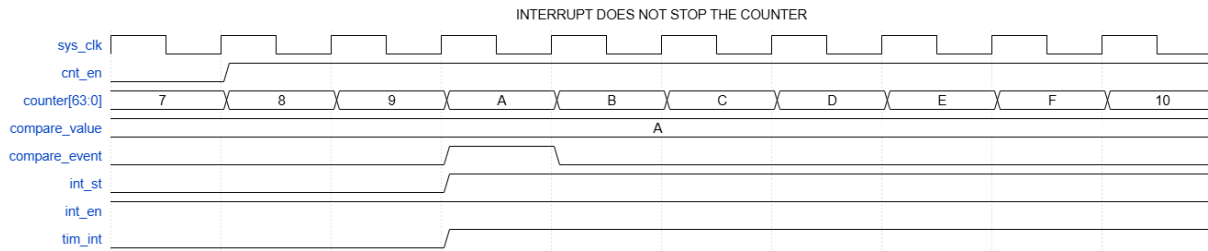


Figure 4-19 Interrupt - No stop counter waveform

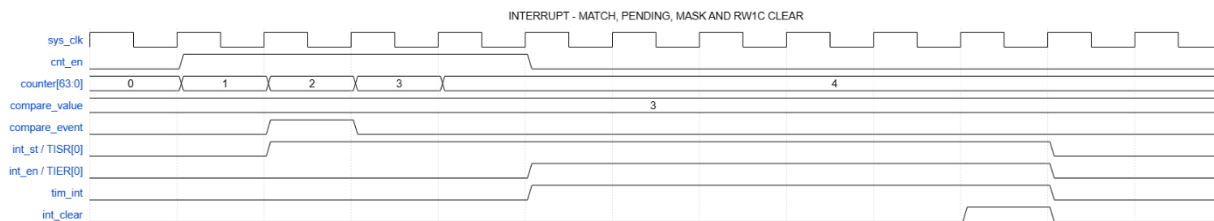


Figure 4-20 Interrupt - Match pending, masking and RW1C clear waveform

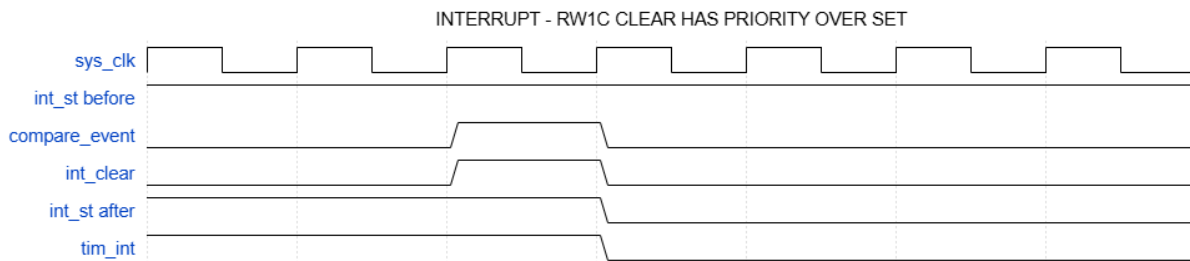


Figure 4-21 Interrupt - RW1C clear priority waveform

4.5. Regression Result

The complete set of directed testcases was executed as part of the final regression to ensure that modifications introduced during RTL debugging or coverage closure did not compromise previously verified functionality. This step is particularly critical given the extensive interaction among design features through shared transaction and control paths. For instance, adjustments to APB completion logic may influence register updates, changes to counter-enable generation may affect interrupt timing, and modifications to TCR error handling may alter timer-disable behavior.

The final regression outcome was:

- Total directed testcases : 15
- PASS : 15
- FAIL : 0

The release checklist specifies that achieving 15/15 PASS constitutes the formal regression acceptance criterion. This result demonstrates that all directed verification scenarios in the final testcase suite completed successfully under the automated checking environment. Consequently, the regression provides strong functional evidence that the final RTL implementation remains consistent across reset, APB protocol, register access, counter operation, debug halt, interrupt handling, and error-management requirements.

In addition, the RTL implementation was verified against the instructor's golden reference model, producing the following outcome:

- Total golden model testcases : 13
- PASS : 13
- FAIL : 0

This result confirms that the RTL design provides strong evidence of conformance for the exercised golden scenarios to the required standards. The remaining task is to complete coverage analysis to ensure that verification achieves comprehensive closure, with all meaningful coverage points exercised to reach 100%.

4.6. Code Coverage Result

Code coverage was assessed as a complementary measure of verification completeness. In accordance with the Final Project methodology, the coverage review encompassed five categories: statement coverage, branch coverage, expression/FEC coverage, condition coverage, and toggle coverage. The final VPLAN defines explicit coverage requirements (COV-001 through COV-005) for each category, with closure targets specified either as complete coverage or as technically justified exclusions where points are proven unreachable by design.

Coverage closure was achieved through detailed analysis of the coverage report to identify remaining uncovered RTL behavior. When an uncovered point represented a valid but untested scenario, additional directed stimulus was introduced. The dedicated testcase `tc_coverage_closure` was employed to exercise residual coverage holes not naturally activated by the main regression. In certain cases, RTL expressions were refactored—without altering functional behavior—to improve observability and enhance the meaningfulness of condition and expression coverage.

It is important to emphasize that coverage results must be interpreted in conjunction with functional checking. Executing a statement only demonstrates that the RTL was reached; it does not confirm correctness of behavior. Therefore, the final verification evidence integrates automated checker results with coverage analysis to provide a comprehensive validation framework.

A suitable final report statement is: Coverage was evaluated using statement, branch, expression/FEC, condition, and toggle coverage. The directed testcase `tc_coverage_closure` was added to exercise remaining meaningful RTL coverage holes. Final coverage evidence is provided by the generated coverage report and corresponding coverage summary.

The actual numerical coverage percentage should be taken directly from the final generated report and presented together with the corresponding screenshot or summary table. Passing tc_coverage_closure confirms successful testcase execution, but does not by itself establish a specific numerical coverage percentage.

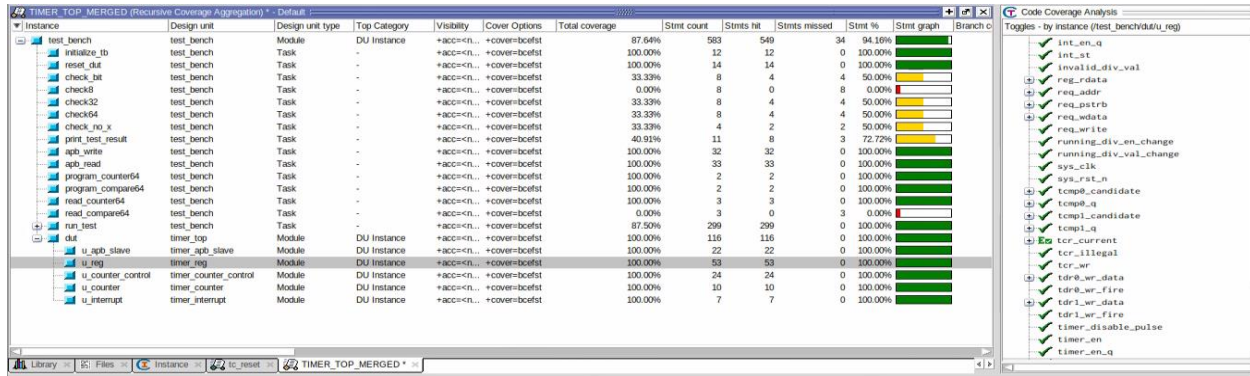


Figure 4-22 Coverage check 1

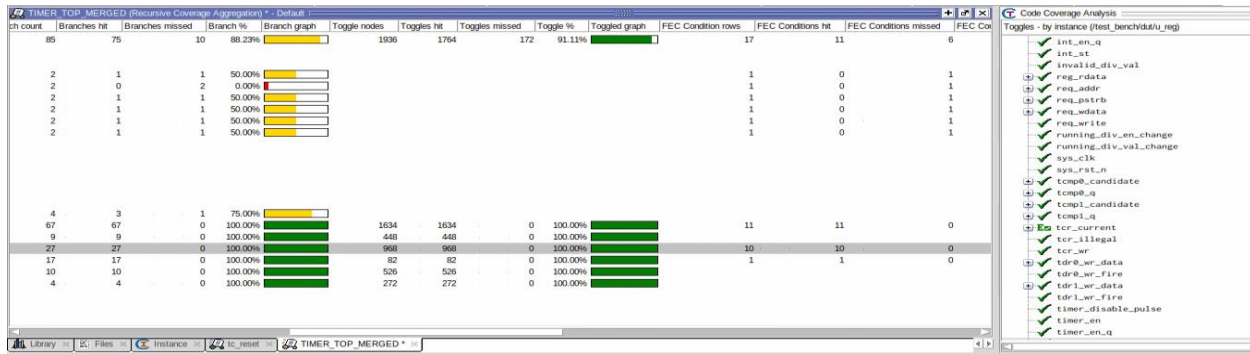


Figure 4-23 Coverage check 2

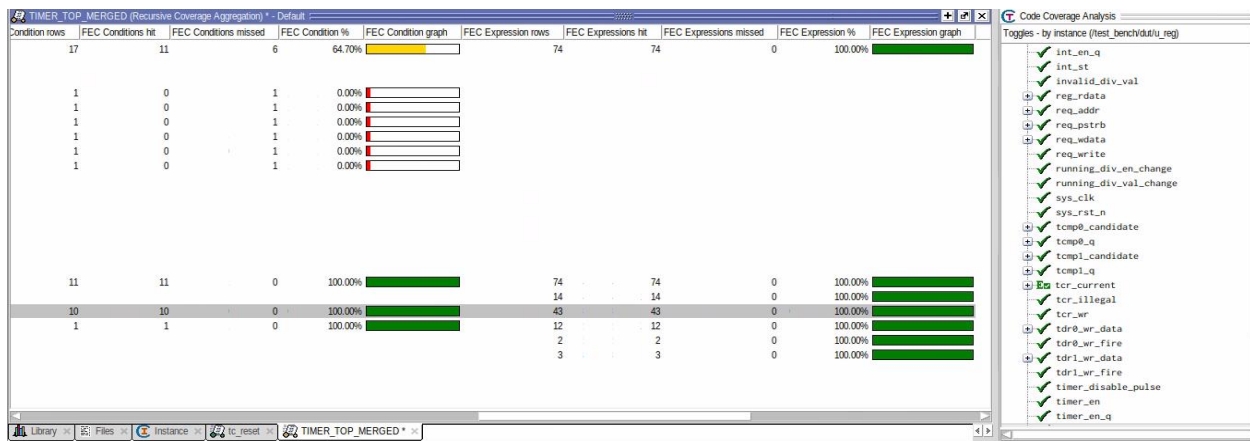


Figure 4-24 Coverage check 3

5. DESIGN CONCLUSION

The Advanced Level Timer IP was successfully completed as a full RTL design and verification project, developed in strict accordance with the functional requirements defined for the Final Project. The implemented architecture integrates a 64-bit count-up timer, APB slave interface, programmable counting control, interrupt generation, debug-halt support, byte-write access, wait-state handling, and prohibited-access detection. These features are organized within a modular structure comprising the APB Slave, Register, Counter Control, Counter, and Interrupt blocks. Clear ownership of state was maintained across modules, ensuring that each functional state is governed by a single block, thereby reducing risks of multiple drivers, duplicated storage, or ambiguous update behavior.

The final design supports both default and programmable counting modes. In default and divider-bypass configurations, the main counter increments on every active system-clock cycle, while programmable divider values from 1 to 8 yield counting periods ranging from 2 to 256 cycles. The divider mechanism is implemented through synchronous clock-enable generation rather than derived clocks, preserving a single `sys_clk` domain throughout the design. The 64-bit counter supports software programming via TDR0/TDR1, low-to-high carry propagation, natural wrap-around, timer-disable clearing, and deterministic update priority across reset, disable, software write, hardware increment, and hold conditions.

The APB interface meets all Advanced Level requirements, including request capture, enforcement of exactly one wait state, byte-level write access through PSTRB, and slave-error response via PSLVERR. Register updates are decoupled from protocol completion through `wr_complete`, `access_error`, and `wr_commit` mechanisms, ensuring that prohibited accesses may complete at the protocol level without altering internal state. TCR legality checking is performed after byte-lane merging, preventing unselected raw data from generating false errors and guaranteeing atomic rejection of illegal transactions.

The interrupt subsystem implements full 64-bit comparison between the current counter value and the programmed compare value. A sticky pending state (`int_st`) records interrupt conditions, while `int_en` governs propagation to the external `tim_int` output. Software clearing is supported via RW1C semantics, with clear priority defined over simultaneous compare matches. Debug functionality is implemented through `halt_req`, `dbg_mode`, and `halt_ack`; when a valid halt condition is asserted, both the main counter and internal divider state are held, allowing counting to resume from the preserved divider phase without altering the configured period.

Verification was conducted using a structured VPLAN, reusable APB testbench tasks, automated checkers, directed testcases, regression, and code-coverage analysis. The final regression comprised 15 directed testcases covering reset, APB protocol, register access, byte writes, error handling, 64-bit counter behavior, counting modes, interrupt operation, debug halt, reserved behavior, reset corner cases, atomic TCR error handling, and coverage closure. All 15 testcases completed with PASS status, providing strong evidence that the RTL implementation satisfies both functional and protocol requirements under normal and corner-case conditions.

In summary, the completed Timer IP fulfills the principal design and verification objectives of the Advanced Level Final Project. By combining a well-defined specification, modular RTL architecture,

deterministic control priorities, requirement-based verification, regression testing, and coverage-driven refinement, the project demonstrates a complete design-and-verification flow—from requirement analysis and micro-architecture development through RTL implementation, functional validation, coverage review, and final release preparation.